WOJSKOWA AKADEMIA TECHNICZNA
im. Jarosława Dąbrowskiego
WYDZIAŁ CYBERNETYKI



PRACA DYPLOMOWA
STUDIA I°

Temat pracy: **SYSTEM OCHRONY PRZED WYBRANYMI ATAKAMI SIECIOWYMI Z WYKORZYSTANIEM REGUŁ IRULES**

INFORMATYKA
(kierunek studiów)

INTERNETOWE TECHNOLOGIE MULTIMEDIALNE
(specjalność)

Dyplomant:

Daniel KRZEWIŃSKI

Promotor:

mgr inż. Krystian WOJDYGOWSKI

Warszawa 2022

OŚWIADCZENIE

Wyrażam zgodę / ~~nie wyrażam~~ ~~zgody~~*
na udostępnianie mojej pracy w czytelni Archiwum WAT.

Dnia 29.05.2022r.

Krzewiński Daniel
(podpis)

**Niepotrzebne skreślić*

Spis treści

Wstęp 5
Rozdział I. Opis i charakterystyka zagrożeń aplikacji internetowych 7
I.1. Charakterystyka aplikacji internetowych 7
I.2. Organizacja OWASP 8
I.3. Zagrożenia aplikacji internetowych 8
Rozdział II. Opis rozwiązania F5 BIG-IP i dostępnych modułów 21
II.1. Firma F5 21
II.2. Charakterystyka produktów firmy F5 21
II.3. Opis produktów z grupy *Traffic Optimization* 22
Rozdział III. Opracowanie projektu systemu zabezpieczeń 25
III.1. Opis istniejącej infrastruktury sieci komputerowej 25
III.2. Założenia projektowe 25
III.3. Wymagania funkcjonalne 26
III.4. Wymagania niefunkcjonalne 26
Rozdział IV. Opracowanie projektu systemu 27
IV.1. Testy penetracyjne aplikacji internetowej przed wdrożeniem zabezpieczenia 27
IV.2. Projekt systemu ochrony 31
Rozdział V. Implementacja systemu 33
V.1. Konfiguracja maszyny wirtualnej 33
V.2. Konfiguracja wstępna systemu 33
V.3. Konfiguracja polityk bezpieczeństwa 34
Rozdział VI. Testowanie systemu ochrony 40
VI.1. Testy penetracyjne zabezpieczonej aplikacji internetowej przy wykorzystaniu aplikacji OWASP Zap 40
VI.2. Przeprowadzenie ataku Cross-Site Scripting 41
VI.3. Przeprowadzenie ataku SQL Injection 42
VI.4. Przeprowadzenie ataku Denial of Service 43
Podsumowanie ..... **Błąd! Nie zdefiniowano zakładki.**
Bibliografia 46

Spis rysunków 47
Załączniki 48

Z komentarzem [WK1]: Czy tu przypadkiem podrozdział 1.1 nie powinien być tak oznaczony, zamiast I.1? Tak samo w pozostałych przypadkach. Proszę zapoznać się z wytycznymi ze strony:
https://www.wcy.wat.edu.pl/pl/studenci/dyplomanci

Z komentarzem [KD2R1]: Dotychczas tak było, z tego co widziałem to zmieniły się wymagania od 2022 roku.
Prace przekopiowałem do gotowego szablonu, który został udostępniony pod tym adresem:
https://wcy.wat.edu.pl/pl/studenci/dyplomanci

Czyli adres jest prawie ten sam co Pan wysłał, bez "www", bardzo dziwne, że pod tymi adresami są całkiem różne informacje, chyba tak nie powinno być, bo informacje są sprzeczne.

Wstęp

Wszechobecny postęp technologiczny stał się częścią życia wszystkich ludzi. Prawie każda dziedzina życia uległa postępowi na tyle, że teraz ciężko sobie ją wyobrazić bez nowoczesnych technik. Społeczeństwo coraz bardziej otwiera się na nowe technologie, które stają się codziennością oraz nieodłącznym elementem życia. Ich zdecydowana większość wymaga stałego połączenia z Internetem do poprawnego działania.

Wraz ze wzrostem liczby osób, które aktywnie korzystają z sieci komputerowych, wzrasta również zainteresowanie aplikacjami internetowymi. W ostatnich latach zdecydowanie zauważalny jest fakt, że odchodzi się od aplikacji instalowanych na komputerze użytkownika, na rzecz aplikacji internetowych. Głównym czynnikiem jest dostęp do omawianych aplikacji z dowolnego miejsca oraz przy użyciu dowolnego urządzenia posiadającego połączenie z Internetem. Dzięki łatwości implementacji i utrzymania aplikacji internetowych, są one szczególnie popularne w szeroko rozumianych rozwiązaniach biznesowych, w tym w sieciach intranetowych zarówno małych jak i globalnych przedsiębiorstwach.

Z komentarzem [WK3]: Po zastosowaniu odpowiedniego formatu, trzeba wszystkie "sieroty" poprzenosić do nowego wiersza za pomocą "twardej spacji".

Zmiany do zastosowania w całej pracy.

Rozwój aplikacji internetowych wiąże się również z coraz to nowszymi atakami i zagrożeniami, na które może być podatna tego typu aplikacja. Istotnym elementem jest zagwarantowanie bezpieczeństwa podatnej aplikacji. Biorąc to pod uwagę, konieczne jest zaprojektowanie oraz wdrożenie dedykowanego systemu zabezpieczeń, który zagwarantuje jej bezpieczeństwo i stabilność działania, pomimo występujących w niej np. błędów implementacyjnych.

Celem pracy jest zapoznanie się z rozwiązaniami dotyczącymi bezpieczeństwa aplikacji firmy F5, zaprojektowanie systemu zabezpieczeń oraz jego implementację wykorzystując rozwiązanie firmy F5, w tym skrypty *iRules*.

Z komentarzem [WK4]: Trzeba ujednolicić w całej pracy zapisy kursywą. Zazwyczaj stosowaliśmy ją w pracy do nazw m.in. do zapisu *iRules, DoS, BIG-IP* itd. Trzeba to ujednolicić w całej pracy.

Zakres pracy został podzielony na sześć rozdziałów. Pierwsze dwa rozdziały skupiają się na aspekcie teoretycznym, a pozostałe cztery rozdziały mają charakter badawczy.

Pierwszy rozdział zawiera określenie czym jest aplikacja internetowa, przedstawia organizację OWASP i jej publikację, a także zawiera opis i charakterystykę grup ataków sieciowych wyszczególnionych w artykule „OWASP Top 10” oraz ataki *Denial of Service*.

Rozdział drugi skupia uwagę na firmie F5, zawiera opis rozwiązań oraz produktów tej firmy, w tym *BIG-IP*.

Rozdział trzeci zawiera analizę fragmentu istniejącej infrastruktury teleinformatycznej, w której zostanie wdrożone rozwiązanie. W tym rozdziale zostały również wyszczególnione wymagania funkcjonalne oraz niefunkcjonalne na wdrażany system.

W rozdziale czwartym zostały przeprowadzone testy podatności aplikacji internetowej, która ma zostać zabezpieczona. W ramach testów wykorzystano dedykowane narzędzie skanujące oraz przeprowadzono testy manualne.

Rozdział piąty opisuje proces wdrożenia dedykowanego rozwiązania firmy F5 wewnątrz istniejącej sieci teleinformatycznej oraz dostosowania systemu zabezpieczeń z wykorzystaniem skryptów *iRules*.

W rozdziale szóstym wykonano testy sprawdzające poprawność implementacji systemu zabezpieczeń oraz przedstawiono wyniki przeprowadzonych badań.

Rozdział I. Opis i charakterystyka zagrożeń aplikacji internetowych

I.1. Charakterystyka aplikacji internetowych

Rozważania na temat aplikacji internetowych należy rozpocząć od możliwie precyzyjnego określenia terminu „aplikacja internetowa", który odzwierciedla rzeczywiste znaczenie tego typu aplikacji. Według wielu dostępnych źródeł, aplikacją internetową możemy nazwać program komputerowy osadzony na serwerze, który komunikuje się z urządzeniem użytkownika końcowego za pomocą sieci komputerowej. Do nawiązywania i przetwarzania tego typu komunikacji, wykorzystywane są przeglądarki internetowe.

Rozważając tematykę aplikacji internetowych za istotne uznaje się wskazanie różnic, jakie występują między aplikacją internetową a witryną (stroną) internetową. Jako podstawową różnicę najczęściej wyróżnia się funkcjonalność oferowaną przez ww. rozwiązania. Głównym elementem składowym strony internetowej są statyczne treści tekstowe, bądź wizualne, które mają na celu przekazanie pewnej informacji użytkownikowi. Taka witryna ma charakter informacyjny i nie jest nastawiona na interakcję z użytkownikiem, w przeciwieństwie do aplikacji internetowych. Działania użytkownika, wykonywane w ramach interakcji z aplikacją internetową, często powodują zmianę wyświetlanych treści na ekranie, a niekiedy pociągają za sobą inne działania, które są wykonywane na serwerze. Wśród nich można wskazać np. zapisanie się do biuletynu informacyjnego, dokonanie płatności on-line czy rezerwacja biletów. Do podstawowych funkcjonalności aplikacji internetowych należą również tzw. operacje *CRUD* (od ang. *Create*, *Read*, *Update*, *Delete*). W ramach operacji z rodziny *CRUD* użytkownik może utworzyć, odczytywać, aktualizować oraz usuwać dane z pamięci trwałej.

W dzisiejszych czasach aplikacje internetowe, posiadają znaczną przewagę nad aplikacjami typu desktop. Pierwszą i najważniejszą ich zaletą jest mobilność. Użytkownik ma możliwość obsługiwać aplikację internetową z dowolnego miejsca na świecie. W tym celu potrzebne jest niemalże dowolne urządzenie z zainstalowaną przeglądarką internetową oraz dostępem do sieci Internet. Z punktu widzenia aplikacji nie ma większego znaczenia w jakim systemie operacyjnym zostanie uruchomiona. Aplikacje internetowe nie wymagają również instalacji na urządzeniu użytkownika, a co za tym idzie, aktualizacje oprogramowania nie wymagają od użytkownika poczynania żadnych dodatkowych kroków.

Kontynuując rozważania na temat aplikacji internetowych za istotne uznaje się poruszenie kwestii ich bezpieczeństwa. Szybki rozwój technologii oraz sieci komputerowych powoduje również wzrost zagrożeń w obszarze bezpieczeństwa sieciowego. Wpływ na to mają systematycznie pojawiające się nowe typy i sposoby ataków oraz publikowane informacje na temat wykrytych luk i podatności wszelkiego rodzaju oprogramowania. Bezpieczeństwo sieciowe jest niezwykle ważne, ponieważ

systemy informatyczne znajdują zastosowanie w niemalże wszystkich dziedzinach gospodarki, a wiele obszarów biznesowych jest zależnych od globalnej sieci Internet. Należy mieć na uwadze, iż potencjalny atak cybernetyczny, w wyniku którego przechwycone zostaną dane, w szczególności o charakterze wrażliwym, może być przyczyną poważnych konsekwencji zarówno wizerunkowych jak i biznesowych dla dotkniętego atakiem przedsiębiorstwa.

I.2. Organizacja OWASP

Przemyślenia na temat bezpieczeństwa aplikacji internetowych należałoby zacząć od dokładnego określenia oraz scharakteryzowania najpoważniejszych zagrożeń, na które podatne są aplikacje internetowe. W wielu źródłach przytaczane są artykuły oraz prace publikowane przez organizację *OWASP* (ang. *Open Web Application Security Project*).

OWASP to społeczność internetowa, powstała w 2001 roku, która publikuje artykuły, metodologie, narzędzia oraz technologie w zakresie bezpieczeństwa aplikacji internetowych. Wszystkie zasoby publikowane przez fundację *OWASP* dostępne są w ramach licencji *FLOSS* (ang. *free/libre and open-source software*) co w uproszczeniu oznacza, iż każdy ma możliwość używania, kopiowania, badania oraz zmiany oprogramowania w dowolnym zakresie.

Fundacja *OWASP* jednoczy osoby, które dzielą się z innymi swoją wiedzą z zakresu bezpieczeństwa oprogramowania. Charytatywne działanie organizacji utwierdza, że publikowane przez nich prace cechują się neutralnością, nie są sponsorowane przez żadną firmę, a co za tym idzie nikt nie wpływa na treść merytoryczną publikowanych materiałów.

I.3. Zagrożenia aplikacji internetowych

Najpopularniejszym artykułem przygotowywanym przez organizację *OWASP* jest „*OWASP Top 10*”, w którym przedstawiane jest 10 najpoważniejszych rodzajów zagrożeń aplikacji internetowych. Najnowsza wersja omawianego artykułu opublikowana została w 2021 roku. Autorzy publikacji sklasyfikowali zagrożenia aplikacji internetowych w ramach następujących grup:

- *Broken Access Control,*
- *Cryptographic Failures,*
- *Injection,*
- *Insecure Design,*
- *Security Misconfiguration,*
- *Vulnerable and Outdated Components,*
- *Identification and Authentication Failures,*
- *Software and Data Integrity Failures,*
- *Security Logging and Monitoring Failures,*

- *Server-Side Request Forgery.*

Mając na uwadze temat niniejszej pracy, za stosowne uznaje się przedstawienie najważniejszych cech dla każdej grupy wymienionej w ramach zestawienia „*OWASP Top 10*".

I.3.1. Zagrożenia z grupy *Broken Access Control*

Zagrożenia z grupy *Broken Access Control* cechują się możliwością nieuprawnionego ujawniania informacji, modyfikacji lub zniszczenia wszystkich danych. Do tej grupy zagrożeń należy też korzystanie z funkcji aplikacji, które zgodnie z założeniami nie są przeznaczone dla użytkownika. Najpopularniejsze podatności z grupy *Broken Acces Control* to:

- omijanie kontroli dostępu poprzez modyfikację adresu URL, stanu aplikacji wewnętrznej lub strony HTML;
- zezwolenie na przeglądanie lub edytowanie cudzego konta poprzez podanie jego unikalnego identyfikatora;
- podniesienie uprawnień – działanie na uprawnieniach użytkownika bez uwierzytelnienia, działanie jako administrator z poziomu konta zwykłego użytkownika;
- manipulowanie metadanymi, w tym także manipulowanie tzw. tokenem kontroli dostępu *JWT* (ang. *JSON Web Token*).

W literaturze przedmiotu można znaleźć informacje świadczące o tym, iż prawie każdy serwer WWW i aplikacja internetowa jest podatna na co najmniej jedno zagrożenie z grupy *Broken Access Control*. W zależności od zasobów, do których atakujący uzyska dostęp, skutki mogą być bardzo szkodliwe. Do takich skutków możemy zaliczyć:

- ujawnienie nieautoryzowanych treści – atakujący może przeglądać, modyfikować, a nawet usuwać poufne dane, zmniejszając wydajność systemu, dostępność oraz obniżając reputację firmy;
- eskalacja uprawnień – atakujący próbują wykorzystać luki w celu uzyskania dostępu do zasobów, które powinny być niedostępne dla normalnego użytkownika. Najczęściej hakerzy próbują przejąć kontrolę nad jak największą liczbą kont użytkowników. Poprzez podniesienie uprawnień atakujący jest w stanie wykraść dane użytkownika lub wdrożyć złośliwe oprogramowanie, które może uszkodzić cały tzw. ekosystem aplikacji;
- ataki *DDOS* (ang. *Distributed Denial of Service)* – mając dostęp do wielu kont użytkowników, atakujący mogą wywołać awarię systemu poprzez wysyłanie wielu jednoczesnych żądań w bardzo krótkich interwałach. Ponadto atakujący są w stanie wgrywać złośliwe oprogramowanie na serwer aplikacyjny, które powoduje niedostępność właściwej aplikacji internetowej lub całego serwera, na którym jest ona uruchomiona.

W celu zmniejszenia ryzyka wystąpienia ataku, warto zastosować się do zaleceń zdefiniowanych przez organizację *OWASP*. Jednym z kluczowych sposobów jest zaimplementowanie jednorazowo kontroli dostępu, a następnie używanie jej ponownie w całej aplikacji. Istotnym elementem jest fakt, że kontrole dostępu powinny wymuszać własności rekordu. Użytkownik nie może być w stanie tworzyć, czytać, aktualizować lub usuwać dowolnego rekordu.

I.3.2. Zagrożenia z grupy *Cryptographic Failures*

Kolejną grupą zagrożeń jest *Cryptographic Failures*. Na zagrożenia z tej grupy są głównie narażone wrażliwe dane, które zostały nieodpowiednio zabezpieczone. Do takich danych możemy zaliczyć:

- hasła,
- numery kart kredytowych,
- dokumentację zdrowotną,
- dane osobowe,
- inne dane, które wymagają dodatkowej ochrony i podlegają przepisom dotyczącym prywatności, np. RODO.

Jako powód potencjalnego wycieku takich danych, organizacja *OWASP* wskazuje związek z błędami na poziomie kryptograficznym lub całkowitym brakiem implementacji takich mechanizmów.

Celem poprawnego zabezpieczenia aplikacji internetowych przed atakami z omawianej grupy zagrożeń należy w pierwszym etapie rozpoznać, w jaki sposób dane są podatne na ujawnienie. Należy więc określić czy potencjalne ujawnienie następuje w stosunku do danych będących w ruchu (transmitowanych np. w sieci teleinformatycznej) czy danych będących w spoczynku (składowanych np. na serwerach wymiany plików).

Dane są często transportowane przez sieć do innych aplikacji lub serwerów. Dane w ten sposób przesyłane są podatne na ataki w momencie przekazywania ich przez niechronione protokoły transmisji lub momencie przekazywania ich do *API* (ang. *Application Programming Interface*). Jednym z ataków, który ma na celu przechwycenie ruchu sieciowego jest atak typu *Man-in-the-Middle*. Za pomocą tego ataku, atakujący jest w stanie przechwycić cały ruch sieciowy, w tym dane, które są przesyłane między węzłami, np. dane logowania użytkownika.

Dane w spoczynku, czyli takie, które są przechowywane w systemie, są uważane za mniej podatne na ataki. Z reguły są to dane bardziej wartościowe, niż dane przesyłane w ruchu sieciowym. Do wykradnięcia danych, atakujący używają zazwyczaj złośliwego oprogramowania. Przykładem takiego oprogramowania jest koń trojański lub robak komputerowy. Za jego pomocą atakujący jest w stanie uzyskać dostęp do informacji przechowywanych w plikach poza obszarami dostępu.

W celu zabezpieczenia aplikacji internetowej przed atakami z grupy *Cryptographic Failures*, autorzy dokumentu *„OWASP Top 10”* zalecają, aby twórcy aplikacji zastosowali się do niżej wymienionych punktów:

- nie należy przechowywać niepotrzebnych poufnych danych oraz usuwać dane tak szybko jak będzie to możliwe,
- należy szyfrować wszystkie poufne dane w spoczynku,
- należy stosować aktualne i silne algorytmy, protokoły i klucze,
- należy szyfrować wszystkie przesyłane dane za pomocą bezpiecznych protokołów, np. *TLS,*
- nie należy stosować starych i niezabezpieczonych protokołów komunikacji, takich jak *FTP* oraz *SMTP* do przesyłania poufnych danych,
- hasła należy przechowywać w postaci zaszyfrowanej, do szyfrowania należy wykorzystać silne algorytmy, np. takie jak *Argon2*, *bcrypt* czy *PBKDF2,*
- klucze powinny być losowo kryptograficznie generowane oraz przechowywane w pamięci za pomocą tablicy bajtów.

I.3.3. Zagrożenia z grupy *Injection*

W przypadku zagrożeń z grupy *Injection*, ataki przeprowadzone na aplikację internetową cechują się przekazywaniem niezweryfikowanych danych do aplikacji oraz zmuszaniem aplikacji do wykonywania poleceń wydawanych przez atakującego. Jednym z najczęściej wykonywanych ataków z tej grupy jest *SQL Injection*. Atak ten polega na wykorzystaniu luk w zabezpieczeniach i przekazaniu dalej spreparowanego kodu tak, aby zmodyfikowane zapytanie *SQL* zostało wykonane na serwerze i przyniosło oczekiwane skutki. Zmodyfikowane zapytanie *SQL* może pobierać inne rekordy z bazy danych, wykonywać dodatkowe zapytania, nawet takie jak *”DROP TABLE table_name”* (usunięcie całej zawartości tabeli SQL). Za pomocą zapytania *SQL* możliwe jest także przeprowadzenie ataku *DOS* (ang. *Denial of Service)*.

Ataki, które mają na celu zmodyfikowanie zapytań *SQL* są najczęstszym przypadkiem ataków z grupy *Injection*. Należy zwrócić uwagę, że oprócz wspomnianego ataku, występują też inne ataki z tej grupy, chociaż nie są tak popularne. Wśród pozostałych ataków wyróżnia się:

- *Cross-Site Scripting (XSS)* – atak ma na celu wstrzykiwanie złośliwego skryptu (zazwyczaj w języku *JavaScript*) do witryn internetowych;
- *Code Injection* – atak polega na wstrzyknięciu do aplikacji kodu, który zostanie wykonany przez serwer aplikacyjny;
- *Command Injection* – atak polega na wstrzyknięciu kodu (polecenia systemowego), który zostanie wykonany na poziomie systemu operacyjnego serwera;
- *CCS Injection* – atakujący wysyła nieprawidłowe sygnały sesji uwierzytelniania między serwerem a klientem. Taki atak umożliwia przejęcie kluczy szyfrowania,

uzyskanie dostępu do komunikacji między serwerem a klientem oraz ewentualnie dokonanie kradzieży tożsamości.

Mając na uwadze dotychczasowe rozważania, ataki z grupy *Injection* mogą za sobą nieść poważne, czasem nawet nieodwracalne szkody. Atakujący są w stanie uzyskać dostęp do całej bazy danych aplikacji, z której mogą pobrać wszystkie dane lub je dowolnie modyfikować, w tym usuwać. Za pomocą ataków *XSS*, nieuprawniona osoba jest w stanie dokonać kradzieży tzw. plików *Cookie*, zmienić ustawienia użytkownika, a nawet przejąć jego sesję. Mając na uwadze powyższe, ważne jest, aby aplikacja internetowa była należycie zabezpieczona przed tą grupą ataków.

W celu zabezpieczenia aplikacji internetowych przed atakami *Injection* warto jest zapoznać się z informacjami, które zostały zaprezentowane w ramach dokumentu „*OWASP Top 10”.* Jedną z podstawowych metod jest korzystanie z bezpiecznego interfejsu *API*, który ogranicza konieczność korzystania z interpretera, zapewnia sparametryzowany interfejs lub korzysta z narzędzi do relacyjnego mapowania obiektów (*ORM*). Kolejnym sposobem na zabezpieczenie się przed omawianymi atakami jest utworzenie listy dozwolonych danych wejściowych użytkownika, czyli tzw. białej listy. Ponadto zaleca się korzystanie z procedur, które są składowane w bazie danych, zamiast wywoływania bezpośrednich zapytań *SQL*. Korzystanie z procedur znacząco obniża szanse na przeprowadzenie udanego ataku *SQL Injection*.

I.3.4. Zagrożenia z grupy *Insecure Design*

Kolejna grupa zagrożeń *Insecure Design* koncentruje się na zagrożeniach aplikacji wynikających z błędów, które powstały podczas projektowania, oraz na błędach związanych z architekturą aplikacji internetowych. Organizacja *OWASP* wyjaśnia, że zagrożenia z tej grupy różnią się od zagrożeń wynikających z niedociągnięć w implementacji. Dobrze zaimplementowany, ale niezabezpieczony projekt wciąż może być podatny na ataki z omawianej grupy, ponieważ podczas projektowania np. nie przewidziano implementacji mechanizmów kontroli bezpieczeństwa.

Jednym z błędów, które zaliczymy do grupy *Insecure Design,* jest przykład przytoczony przez autorów publikacji *„OWASP Top 10”*. Dotyczy on mechanizmu resetowania hasła wykorzystującego kontrolne pytania i odpowiedzi. Często, odpowiedzi na wybrane pytania można wyszukać w profilach społecznościowych danego użytkownika lub są z reguły znane. Nie jest ważne w jaki sposób zostanie zaimplementowany przedmiotowy mechanizm - jest on obarczony znacznym ryzykiem ze względu na swoją naturę.

Implementację bezpiecznej aplikacji internetowej należy zacząć od poprawnego i bezpiecznego projektowania. W tym celu organizacja *OWASP* przedstawia kilka propozycji, które powinny być stosowane w trakcie projektowania aplikacji, takie jak:

- ustanowienie i korzystanie z bezpiecznego cyklu rozwoju oprogramowania,
- korzystanie z istniejących bibliotek bezpiecznych wzorców projektowych,

- używanie modelowania zagrożeń do krytycznego uwierzytelniania kontroli dostępu, logiki biznesowej czy przepływu kluczy,
- integrowanie kontroli wiarygodności na każdym poziomie aplikacji, od warstwy *Frontend* do warstwy *Backend*,
- korzystanie z testów jednostkowych i integracyjnych, które mogą sprawdzić czy oprogramowanie jest odporne na zagrożenia,
- ograniczenie zużycia zasobów przez użytkownika lub usługę.

I.3.5. Zagrożenia z grupy *Security Misconfiguration*

Grupa zagrożeń *Security Misconfiguration*, dotyczy przeprowadzania ataków wykorzystujących luki w zabezpieczeniach, które są spowodowane błędną ich konfiguracją lub całkowitym brakiem tychże zabezpieczeń. Należy zauważyć, że błędy związane z tą grupą mogą wystąpić na dowolnym poziomie aplikacji, a w szczególności na serwerach aplikacyjnych, serwerach WWW, w bazach danych, czy nawet w zainstalowanych maszynach wirtualnych.

Istnieje wiele powodów, które mogą wskazywać, dlaczego aplikacja jest podatna na zagrożenia z omawianej grupy. Jednym z nich jest brak wzmocnienia zabezpieczeń na dowolnym poziomie aplikacji. Wszystkie moduły, które współpracują w ramach aplikacji internetowej, powinny być silnie zabezpieczone i odporne na ataki. Kolejny powód stanowią niepotrzebne funkcje, które są domyślnie włączone, na przykład na serwerze wykorzystującym bazę danych. Niepotrzebnie otwarte porty sieciowe czy włączone usługi mogą stanowić tzw. otwartą „furtkę" dla potencjalnej osoby atakującej.

Jednym z najczęściej doświadczanych problemów jest wyświetlanie użytkownikowi fragmentu stosu wywołań (ang. *Stack trace*), który generowany jest w momencie wystąpienia błędu lub wyjątku po stronie serwera aplikacyjnego. Bezpośrednio ze stosu wywołań atakujący jest w stanie odczytać wiele informacji na temat podatnej aplikacji, w tym m.in. język programowania w jakim została zaimplementowana aplikacja, wersję wykorzystywanego serwera bazy danych, nazwy wykorzystywanych pakietów, klas i metod, a także bezpośrednie ścieżki do plików. Informacje te pomimo, iż wydają się bezużyteczne mogą być wykorzystane do eskalacji ataku. W przypadku gdy aplikacja jest podatna na tzw. *Path Traversal*, znajomość bezpośrednich ścieżek do plików umożliwia atakującemu np. rozpoznanie struktury katalogowej aplikacji.

Kolejnym niebezpiecznym przykładem błędów związanych z niepoprawną konfiguracją systemu jest ujawnianie wersji oprogramowania. Znając wersję wykorzystywanego oprogramowania atakujący jest w stanie ukierunkować swój atak, m.in. w oparciu o powszechnie znane i dostępne narzędzia wykorzystujące podatności występujące w danej wersji oprogramowania. W tym przypadku ważne jest by mieć na uwadze, iż utajnienie samej informacji na temat oprogramowania nie rozwiązuje omawianego problemu. Należy, jeśli to możliwe, jak najszybciej przeprowadzać aktualizacje przestarzałego oprogramowania.

Mając na uwadze powyższe ważne jest, aby wystrzegać się błędów zidentyfikowanych w ramach *Security Misconfiguration*. W tym celu zaleca się skorzystanie m.in. z wymienionych zaleceń:

- należy zabezpieczyć system poprzez rozwijanie i automatyzację procesu wdrażania nowego środowiska. Każde środowisko powinno być identycznie skonfigurowane, ale powinno korzystać z własnych danych uwierzytelniających;
- na swojej platformie, należy usunąć lub nie instalować zbędnych funkcji i nadmiarowego oprogramowania;
- należy wprowadzić przegląd i aktualizację konfiguracji wszystkich uwag dotyczących bezpieczeństwa. Wszystkie znane błędy bezpieczeństwa powinny być jak najszybciej usuwane np. poprzez wgranie stosownej łatki lub zmianę konfiguracji;
- w komunikacji między aplikacją a użytkownikiem należy wykorzystywać stosowne dyrektywy bezpieczeństwa, w tym odpowiednie nagłówki żądań i odpowiedzi.

I.3.6. Zagrożenia z grupy *Vulnerable and Outdated Components*

Kolejną grupą zagrożeń wyspecyfikowaną w dokumencie *„OWASP Top 10”* jest grupa *Vulnerable and Outdated Components*. W poprzedniej wersji artykułu, opublikowanej w 2017 roku, omawiana grupa zagrożeń scharakteryzowana została pod nazwą *Using Components with Known Vulnerabilities*. Zagrożenia tego rodzaju charakteryzują się wykorzystaniem luk w komponentach oprogramowania, związanych z bezpieczeństwem aplikacji. Komponent oprogramowania to część aplikacji, która rozszerza jej funkcjonalność. Do komponentów zaliczamy więc m.in. moduły, pakiety oprogramowania lub API.

Na uwagę zasługuję fakt, że w przypadku oprogramowania modułowego, bezpieczeństwo całego systemu informatycznego, zależy od bezpieczeństwa poszczególnych modułów. System informatyczny może być podatny na ataki jeśli:

- administrator nie zna wersji wykorzystywanych komponentów,
- oprogramowanie jest już niewspierane lub nieaktualne,
- nie zostają wykonywane regularne skanowania i testy mające na celu wykrycie luk w zabezpieczeniach aplikacji.

Mając na uwadze powyższe rozważania możliwe jest przedstawienie przykładowego scenariusza ataku. W ramach opisanych działań atakujący może wykorzystać luki w zabezpieczeniach, co w konsekwencji pozwoli na wykonanie złośliwego kodu na serwerze, na którym osadzono podatną aplikację, poprzez:

1. uzyskanie nieautoryzowanego dostępu do sieci wewnętrznej,
2. uruchomienie narzędzia skanującego w celu zlokalizowania systemów, korzystających z niezałatanych lub nieaktualizowanych komponentów,

3. wykorzystanie wykrytej w komponencie podatności i przeprowadzenie ataku polegającego na wstrzyknięciu złośliwego kodu do serwera aplikacyjnego.

Autorzy publikacji *„OWASP Top 10”* w swoim opracowaniu zaprezentowali zbiór kroków, które należy wykonać, aby upewnić się, że aplikacja internetowa nie jest podatna na zagrożenia omawianej grupy. W tym celu zaleca się:

- usunięcie nieużywanych zależności, w tym niepotrzebnych funkcji i komponentów,
- stałe kontrolowanie używanych wersji komponentów oraz ich zależności (np. przy użyciu dedykowanego narzędzia *OWASP Dependency Check*),
- stałe monitorowanie źródeł poruszających kwestie bezpieczeństwa sieciowego, takich jak *Common Vulnerability and Exposures (CVE)*,
- korzystanie z komponentów pochodzących wyłącznie z oficjalnych źródeł.

Należy mieć na uwadze, że nie istnieje uniwersalny opis procedur, które zagwarantują, że aplikacja jest w pełni zabezpieczona. Należy stale kontrolować stan aplikacji i cyklicznie wykonywać jej testy, aby możliwie najlepiej ją ochronić. Przedstawione kroki stanowią ogólną podstawę do takiego działania. Trzeba mieć na uwadze, że w wykorzystywanym oprogramowaniu może występować wiele podatności, które nie zostały jeszcze zidentyfikowane, a ich wykrycie w przyszłości nie jest wykluczone.

I.3.7. Zagrożenia z grupy *Identification and Authentication Failures*

Zagrożenia z grupy *Identification and Authentication Failures* charakteryzują się wykorzystywaniem luk w zabezpieczeniach związanych z niepoprawną implementacją lub niewystarczającą ochroną tożsamości użytkownika, uwierzytelnianiem lub zarządzaniem sesją. Tego typu zagrożenia mogą występować jeśli w aplikacji:

- dopuszczane są zautomatyzowane ataki, np. *credential stuffing*,
- dopuszczane są ataki typu *brute force*,
- dopuszczane jest korzystanie ze słabych, popularnych haseł, takich jak: „*admin*” lub „*Password1*”,
- przechowywane hasła nie są szyfrowane lub są słabo szyfrowane,
- identyfikator sesji jest ujawniany w adresie URL.

Celem ataków z omawianej grupy jest przejęcie jednego lub kilku kont oraz uzyskanie przez atakującego takich samych uprawnień, jakie posiada atakowany użytkownik. Jeśli atakującemu uda się przejąć konto administratora, uzyska on dostęp do wszystkich funkcji administracyjnych. W zależności od budowy aplikacji i jej przeznaczenia taki incydent może doprowadzić poważnych konsekwencji wizerunkowych, finansowych jak i prawnych.

Istotne jest, aby zmniejszyć ryzyko związane z potencjalną skutecznością przeprowadzonego ataku z omawianej grupy. W tym celu warto zastosować się do zaleceń autorów publikacji *„OWASP Top 10”*. Jej twórcy przedstawiają szereg kroków,

które powinny być przestrzegane, aby uchronić się przed tego typu atakami. Wśród nich wyróżnia się m.in.:

- wdrożenie uwierzytelniania wieloetapowego tam gdzie jest to możliwe,
- wdrożenie mechanizmu sprawdzania słabych haseł w odniesieniu do listy 10 000 najpopularniejszych haseł [5],
- ograniczanie lub opóźnianie kolejnych prób nieudanego logowania, rejestrowanie wszystkich niepowodzeń i ostrzeganie administratorów w przypadku wykrycia ataków typu *credential stuffig* lub *brute force*,
- korzystanie z bezpiecznego, wbudowanego po stronie serwera, menadżera sesji, który po zalogowaniu generuje nowy, losowy identyfikator sesji. Identyfikator sesji nie powinien znajdować się w adresie URL, powinien być bezpiecznie przechowywany i unieważniony tuż po wylogowaniu lub w wyniku bezczynności użytkownika.

I.3.8. Zagrożenia z grupy *Software and Data Integrity Failures*

Kolejną grupę zagrożeń opisanych w publikacji „*OWASP Top 10*" nazwano *Software and Data Integrity Failures*. Zagrożenia tej grupy bazują na błędach dotyczących kodu i infrastruktury aplikacji, które nie chronią przed naruszeniami integralności. Może do tego dojść w przypadku korzystania z oprogramowania pochodzącego z niezaufanych źródeł lub oprogramowania, które zostało naruszone u źródła, w takcie przesyłania lub w pamięci punktu końcowego. Wykorzystując te podatności, atakujący może uzyskać nieautoryzowany dostęp do aplikacji, możliwość wprowadzania złośliwego kodu lub uszkodzenia całego systemu dokonując ataków takich jak *cache poisoning*, *code injection*, *Denial of Service*.

Omawiana grupa zagrożeń obejmuje również podatności wynikające z błędnej implementacji mechanizmu deserializacji. Proces serializacji polega na przekształcaniu obiektów w inny format, w tym binarny lub strukturalny, do którego należą formaty takie jak JSON czy XML. Deserializacja polega na przywróceniu przez aplikację zserializowanych danych do oryginalnej postaci. Wykorzystując niezabezpieczony mechanizm deserializacji, atakujący zazwyczaj modyfikują dane zawarte w obiektach zmieniając logikę działania samej aplikacji. Jeśli atak zakończy się sukcesem, może być przyczynkiem do kolejnych ataków takich jak *Denial of Service* lub *code injection*.

Mając na uwadze powyższe istotne jest, aby wystrzegać się błędów zidentyfikowanych w ramach omawianej grupy. W tym celu zaleca się przestrzeganie określonych reguł, wśród których wyróżnia się:

- używanie podpisów cyfrowych lub analogicznego mechanizmu, który zapewni, że oprogramowanie lub przetwarzane dane pochodzą z oczekiwanego źródła i nie zostały zmodyfikowane,
- upewnienie się, że biblioteki pobierane są z zaufanych repozytoriów,

- upewnienie się, że istnieje proces weryfikacji zmian kodu aplikacji, aby zminimalizować prawdopodobieństwo wprowadzenia do niego złośliwych fragmentów,
- upewnienie się, że żadne serializowane dane nie są wysyłane do klientów w postaci niezaszyfrowanej lub niepodpisanej.

I.3.9. Zagrożenia z grupy *Security Logging and Monitoring Failures*

Następną grupą zagrożeń, na którą zwracają uwagę autorzy publikacji „*OWASP Top 10*", jest *Security Logging and Monitoring Failures*. Warto przy tym zaznaczyć, że nie ma bezpośredniej podatności, która byłaby wynikiem problemów związanych z logowaniem i monitorowaniem. Nie mniej jednak, brak tych mechanizmów lub błędy w ich implementacji mogą mieć bezpośredni wpływ na przejrzystość działania aplikacji oraz brak ostrzegania o incydentach. Do omawianej grupy zagrożeń można zaliczyć przypadki takie jak:

- niezapisywanie udanych i nieudanych prób logowań,
- brak kopii zapasowej logów w przypadku awarii serwera aplikacji przechowującego logi,
- niejasne lub niewłaściwe zapisy dzienników zdarzeń, które nie dostarczają żadnych wartościowych informacji,
- systemy monitorowanie nie są w stanie wykrywać podejrzanych działań lub nie są w stanie alarmować o incydentach w czasie rzeczywistym.

Ważnym elementem każdej aplikacji internetowej jest posiadanie funkcjonalnego systemu monitorowania i rejestrowania, który gromadzi zdarzenia i alarmuje o wszelkich awariach bądź błędach. W przypadku jego braku, istnieje ryzyko, że przez długi czas awarie lub błędy nie zostaną zauważone przez administratorów, co w konsekwencji może doprowadzić do wystąpienia znacznie poważniejszych szkód.

W celu ochrony przed omawianymi błędami zaleca się, aby na etapie implementacji aplikacji internetowych stosować się do określonych zaleceń oraz przestrzegać podstawowe reguły bezpieczeństwa. Wśród nich wyróżnia się m.in.:

- zapewnienie, że logi generowane są w formacie, który może być łatwo wykorzystywany przez rozwiązania klasy SIEM,
- zapewnienie, że logi są prawidłowo kodowane, aby uniknąć ataków z grupy *injection* lub ataków bezpośrednio na systemy monitorowania,
- zachowywanie kopii zapasowej logów,
- upewnienie się, że alarmowanie o podejrzanych zdarzeniach realizowanie jest w czasie rzeczywistym.

I.3.10. Zagrożenia z grupy *Server-Side Request Forgery*

Ostatnia grupa zagrożeń scharakteryzowana w ramach publikacji „*OWASP Top 10*" to *Server-Side Request Forgery*. W ramach tej grupy wyróżnia się zagrożenia polegające

na wykorzystywaniu luk w zabezpieczeniach sieciowych, które umożliwiają atakującemu skanowanie bądź pobieraniu zasobów z sieci lokalnej, w której uruchomiono aplikację internetową.

Atakowana aplikacja może posiadać funkcjonalności związane z danymi dostępnymi pod adresem URL, w tym importowanie tych danych, publikowanie pod podanym adresem lub odczytywanie danych z adresu URL. Atakujący poprzez modyfikację adresu URL może doprowadzić do zmiany sposobu działania danej funkcjonalności.

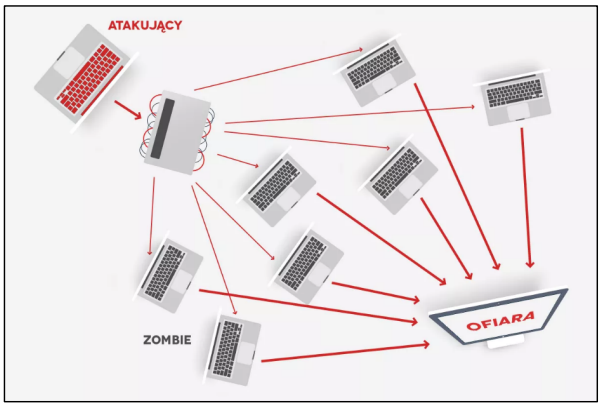
Udany atak może skutkować nieautoryzowanymi działaniami lub uzyskaniem przez atakującego dostępu do danych przetwarzanych w ramach podatnej aplikacji, jak również w innych systemach, z którymi komunikuje się ta aplikacja. W niektórych przypadkach, udane wykorzystanie tego typu podatności może umożliwić atakującemu zdalne wykonywanie poleceń po stronie serwera aplikacyjnego.

W celu ochrony aplikacji przed omawianym typem zagrożeń zaleca się stosowanie tzw. białej listy dozwolonych domen i protokołów, dla zdalnych zasobów pobieranych przez serwer. Ponadto należy też unikać bezpośredniego używania i wykonywania danych wejściowych wprowadzanych przez użytkownika, aby uniemożliwić atakującemu zdalne wykonywanie kodu po stronie serwera.

I.3.11. Zagrożenia z grupy *Denial of Service*

Oprócz grup ataków scharakteryzowanych w ramach artykułu „*OWASP Top 10*”, warto zwrócić uwagę na ataki typu *DOS* (ang. *Denial of Service*). Celem ataku *DOS* jest uniemożliwienie korzystania z maszyny lub zasobu sieciowego. Atak charakteryzuje się ciągłym wysyłaniem żądań HTTP pod konkretny adres IP, które mają na celu przeciążeniu serwera aplikacji lub strony internetowej. Ataki *DOS* blokują dostęp do zasobu sieciowego użytkownikom, którzy chcieli go wykorzystać zgodnie z jego przeznaczeniem.

Jednym z typów omawianych ataków jest *DDOS* (ang. *Distributed Denial of Service*). Polega na atakowaniu ofiary z wielu miejsc jednocześnie, gdzie każda atakująca maszyna ma unikalny adres IP. Do ataku najczęściej wykorzystywane są maszyny, nad którymi przejęto kontrolę za pomocą złośliwego oprogramowania.



Rys. 1. Schemat działania ataku DDoS

Źródło: https://domeny.pl/blog/content/webp-express/webp-images/uploads/2017/10/Przechwytywanie-DDoS-Blog.jpg.webp

Celem ataku *DDOS*, podobnie jak *DOS*, jest przeciążenie atakowanego systemu, co może doprowadzić do całkowitej niedostępności zasobów. Ataki *DDOS* są znacznie trudniejsze do wykrycia i zablokowania niż ataki *DOS*. W przypadku tych drugich wystarcza tzw. „czarna lista", czyli lista adresów IP, które mają zablokowany dostęp do zasobów. W przypadku *DDOS* obrona jest zdecydowanie bardziej utrudniona ze względu na:

- nieograniczoną liczbę źródeł ataku,
- zainfekowane komputery, które wysyłają równolegle atak *DOS* mogą znajdować się na całym świecie,
- odfiltrowanie ruchu sieciowego dla konkretnego adresu IP skutkuje brakiem dostępu dla realnego użytkownika, który nie jest świadomy, że generuje ataki *DOS*.

Oprócz ataków *DDOS*, istnieje jeszcze wiele innych typów tych ataków, a hakerzy ciągle rozwijają swoje ataki, wykorzystując luki w zabezpieczeniach. Kolejnym przykładem ataku typu *DOS* jest *DRDOS* (ang. *Distributed Reflected Denial of Service*) czyli rozproszony odbity atak odmowy dostęp. Do tego ataków wykorzystuje się spreparowane żądania – nadawca zostaje sfałszowany. Odpowiedź na takie zapytania, nie trafia do prawdziwego nadawcy, tylko do sfałszowanego nadawcy. Atakujący wysyła spreparowane żądania do przypadkowych adresów IP, a one wyślą odpowiedź do jednego, atakowanego adresu IP.

Obrona przed atakami typu *DOS* jest niezwykle trudna i problematyczna. Nie istnieje żaden schemat obrony, który zagwarantuje pełną ochronę przed tego typu zagrożeniem. Jednym z kluczowym elementów jest przygotowanie przez organizację, która może być celem ataku, planu reagowania i odnowy po ataku *DOS*. W ramach planu powinny znajdować się instrukcje jak krok po kroku reagować na atak, do kogo zwrócić się w

przypadku wykrycia ataku oraz jakie dany członek organizacji ma obowiązki podczas ataki *DOS*. Poza tym, bardzo ważne jest ciągłe analizowanie oraz monitorowanie ruchu sieciowego. Ruch sieciowy może być kontrolowany przy użyciu zewnętrznego oprogramowania, np. *BIG-IP LTM*. Administratorzy mogą skonfigurować reguły, które tworzą alerty w przypadku wykrycia nietypowego ruchu lub nawet usuwają pakiety sieciowe, które zostaną odfiltrowane jako atak *DOS*.

Rozdział II. Opis rozwiązania F5 BIG-IP i dostępnych modułów

II.1. Firma F5

Zagwarantowanie bezpieczeństwa aplikacji internetowych przed ciągle rozwijającymi się atakami sieciowymi wymaga ciągłej pracy wielu ludzi, ich czujności i badań oraz przede wszystkim szybkiej reakcji. Nie każde przedsiębiorstwo czy osoba prywatna posiada odpowiednią wiedzę i środki na zatrudnianie kilku osób dbających o bezpieczeństwo całej infrastruktury sieciowej. W związku z tym, popularnym rozwiązaniem jest korzystanie z gotowych produktów oferowanych przez firmy, specjalizujące się w bezpieczeństwie sieci komputerowych, w tym aplikacji internetowych.

Jedną ze szczególnie rozpoznawalnych firm jest F5, amerykańska firma technologiczna specjalizująca się w bezpieczeństwie aplikacji, zarządzaniu chmurami, zapobieganiu oszustwom internetowym, dostępności oraz wydajności aplikacji. Firma F5 została założona w 1996r. w USA, a jej siedziba znajduje się w Seattle. Według najnowszych danych, omawiana firma, posiada 75 biur w 43 różnych krajach, w tym w Polsce w Warszawie, a z usług F5 korzysta 48 firm, które znajdują się w rankingu Fortune 50 (lista 50 największych banków, przedsiębiorstw użyteczności publicznej, zakładów ubezpieczeniowych, sprzedawców detalicznych oraz firm transportowych w Stanach Zjednoczonych) oraz takie przedsiębiorstwa jak Microsoft, Oracle, Tesla czy Meta (wcześniej znana jako Facebook).

Oferta F5 początkowo opierała się na rozwiązaniach kontrolerów dostarczania aplikacji (ADC), ale od tego czasu rozszerzyła się na szeroką gamę warstw aplikacji, automatyzacji, usług wielochmurowych i bezpieczeństwa. Pierwszym rozwiązaniem zaprezentowanym przez firmę F5 był Load-Balancer znany jako BIG-IP. Obecnie rodzina produktów BIG-IP obejmuje sprzęt, oprogramowanie modułowe oraz urządzenia wirtualne oparte na systemie operacyjnym F5 TMOS.

II.2. Charakterystyka produktów firmy F5

Portfolio usług oraz produktów oferowanych przez firmę F5 jest bardzo szerokie. Produkty te zapewniają aplikacji internetowej szybkość w działaniu, dostępność oraz bezpieczeństwo w każdym środowisku, czy to lokalnym lub wielochmurowym. Oferowane przez firmę F5 produkty podzielone zostały na 4 kategorie zgodnie z ich przeznaczeniem:

- *Traffic Optimization*,
- *Security*,
- *Cloud Services*,
- *Cloud Software*.

W ramach kategorii *Traffic Optimization* wyróżniamy przede wszystkim produkty należące do rodziny produktów *BIG-IP*. Głównym celem rozwiązań z tej kategorii jest

zrównoważenie obciążenia ruchu sieciowego poprzez kontrolowanie ruchu oraz jego optymalizację.

Kolejną z wymienionych kategorii stanowi *Security*. Bezpieczeństwo jest jedną z podstawowych kwestii poruszanych w ramach projektowania aplikacji internetowej. Nieupoważnione naruszenie oraz wyciek wrażliwych danych może doprowadzić w danej organizacji do poważnych konsekwencji prawnych, wizerunkowych jak i finansowych. Rozwiązania firmy F5 z omawianej grupy, w tym *Access Policy Manager*, *Advanced WAF* (*Web Application Firewall*) czy *DDOS Hybrid Defender*, zapewniają stałą ochronę przed ciągle zmieniającymi się zagrożeniami.

Kategoria *Cloud Services* skupia swoje działanie oparte na modelu oprogramowania jako usługi (ang. *Software as a Service*, *SaaS*). Model umożliwia odbiorcy usług wykorzystywanie aplikacji uruchomionych w chmurze, dostarczanych przez dostawcę usług, w tym przypadku przez firmę F5. Według założeń modelu, obowiązki instalacji, zarządzania, aktualizacji aplikacji internetowej są przenoszone na dostawcę usługi. W przypadku korzystania z *Cloud Services*, przedsiębiorstwo F5 zapewnia ciągłą aktualizację bezpieczeństwa oprogramowania, szybkość w działaniu aplikacji internetowej, dostępność z całego świata, a przede wszystkim ciągłe wsparcie i pomoc techniczną dostępną 24 godziny na dobę, 7 dni w tygodniu.

W ramach ostatniej grupy *Cloud Software*, firma F5 oferuje usługi aplikacyjne dostarczane w oprogramowaniu w chmurze. Do swoich usług, przedsiębiorstwo zalicza takie świadczenia jak:

- rozwijanie i wdrażanie aplikacji w chmurze,
- tworzenie nowych aplikacji oraz migrowanie istniejącego już oprogramowania do chmury,
- zapewnienie szybkości i ciągłości działania aplikacji.

Ponadto firma F5 oferuje też wirtualne wersje produktów *BIG-IP*, takie jak *BIG-IP Local Traffic Manager*, *Advanced WAF* i wiele innych.

II.3. Opis produktów z grupy *Traffic Optimization*

Biorąc pod uwagę temat niniejszej pracy, produkty przydzielone do grupy *Traffic Optimization* zasługują na specjalną uwagę. Zapewniają one zrównoważenie przepływu ruchu sieciowego, jego optymalizację oraz bezpieczeństwo aplikacji internetowych. Produkty z rodziny *BIG-IP* są głównymi rozwiązaniami oferowanymi przez firmę F5. Dzielą się na:

- *application services*,
- *service providers*,
- *platforms*.

Pierwszym produktem z podgrupy *application services* jest *BIG-IP LTM* (ang. *Local Traffic Manager*). To rozwiązanie pomaga dostarczać aplikacje internetowe do użytkowników zapewniając niezawodność, optymalizację oraz przede wszystkim

bezpieczeństwo. Daje również możliwość kontrolowania ruchu sieciowego poprzez odpowiedni dobór miejsca docelowego na podstawie wydajności i dostępności serwera. Kluczowymi zaletami produktu *BIG-IP LTM* są:

- dostarczanie aplikacji internetowych w sposób niezawodny i szybki,
- automatyzacja oraz dostosowanie do wymagań dzięki programowalnej infrastrukturze,
- łatwe wdrażanie oraz zarządzanie aplikacjami internetowymi,
- zabezpieczenie aplikacji internetowej.

Omawiany produkt jest w pełni programowalny. Oznacza to, że oprócz gotowych funkcji, które dostarcza *BIG-IP LTM*, możemy zarządzać ruchem sieciowym zgodnie z potrzebami. Wykorzystywane w tym celu są reguły *iRules*. *iRule* jest skryptem, który pozwala wykorzystywać rozszerzone możliwości *BIG-IP*, które nie są dostępne z poziomu interfejsu graficznego oraz wiersza poleceń. W ramach skryptów *iRules* można przekierowywać ruch sieciowych do konkretnych pul adresowych, a nawet do poszczególnych członków puli. Oprócz wspomnianych wyżej funkcjonalności, z pomocą skryptów *iRules*, istnieje możliwość analizy nagłówków protokołów, takich jak IP, TCP oraz UDP, oraz analiza danych. Użytkownikowi udostępniona jest lista komend gotowych do wykorzystania w ramach skryptu, do których należą takie funkcje jak logowanie czy przekierowywanie ruchu, a nawet modyfikacja przesyłanych danych.

Jednym z modułów, które mogą zostać wdrożone razem z *BIG-IP LTM* jest *BIG-IP ASM* (ang. *Application Security Manager*). Rozwiązanie ASM jest zaporą sieciową skonstruowaną bezpośrednio pod aplikacje internetowe. Chroni je przed znanymi oraz nieznanymi zagrożeniami bezpieczeństwa, w tym:

- atakami *Cross-Site Scripting*,
- atakami *SQL Injection*,
- atakami *DDOS* w warstwie 7,
- przechwytywaniem sesji,
- manipulacją ciasteczkami.

Rozwiązanie *ASM* zostało wyposażone w ponad 2200 sygnatur ataków, stanowią one podstawę do zezwalania lub blokowania ruchu sieciowego. Ten produkt zezwala również na tworzenie własnych sygnatur ataków, dzięki czemu możliwe jest dopasowanie ochrony do własnych potrzeb. Z uwagi na ciągle rozwijające się ataki sieciowe, baza sygnatur jest ciągle aktualizowana.

Drugim produktem w kategorii *application services* jest *BIG-IP DNS* (ang. *Domain Name System*). Odpowiedzialny jest on za dystrybuowanie zapytań DNS. Wybierając odpowiedni serwer DNS brane są pod uwagę takie czynniki jak: warunki w centrum danych i usług w chmurze, lokalizację użytkownika oraz wydajność aplikacji. Ponadto *BIG-IP DNS* oferuje wysoką wydajność usług DNS wraz raportowaniem oraz analizą żądań. Omawiany produkt zapewnia również ochronę przed atakami *DDOS* (ang. *Distributed Denial of Service*) na serwer DNS.

Produkty z kategorii *service providers* nastawione są na zarządzanie zasobami sieciowymi, aby utrzymać wydajność aplikacji na wysokim poziomie. Oprócz tego, mają na celu znajdowanie sposobów optymalizacji sieci. Pierwszym produktem z tej podgrupy jest *BIG-IP Diameter Traffic Management*. Odpowiedzialny jest on za zarządzanie przeciążeniem ruchu w sieci poprzez:

- przyznanie pierwszeństwa ruchu według typu lub urządzenia,
- wskazywanie i usuwanie przyczyn nadmiernego ruchu sieciowego,
- gwarancję wydajności,
- monitorowanie i ochronę elementów sieci.

Kolejnym rozwiązaniem należącym do podgrupy *service providers* jest *BIG-IP Policy Enforcement Manager*. Produkt ten odpowiedzialny jest za zarządzanie ruchem sieciowym poprzez sterowanie w warstwach 4-7 modelu ISO/OSI. Ponadto zapewnia kontrolę zasobów sieciowych oraz generuje szczegółowe raporty na podstawie sposobu wykorzystywania aplikacji internetowej przez klientów. Kluczową zaletą produktu *BIG-IP PEM* jest zmniejszenie ryzyka związanego z przeciążeniem sieci internetowej oraz zapewnienie wydajności aplikacji internetowej poprzez kierowanie ruchu sieciowego w warstwie aplikacji modelu ISO/OSI.

Ostatnią omawianą kategorią jest grupa *platforms*. Firma F5 oferuje gotowe rozwiązania, które umożliwiają wdrożenie aplikacji internetowych w sieci wewnętrznej oraz chmurze. Do omawianej grupy zaliczają się urządzenia sprzętowe *BIG-IP iSeries* oraz *VIPRION*, a do oprogramowania należą produkty *BIG-IP Virtual Edition*.

Obie rodziny produktów, *BIG-IP iSeries* oraz *VIPRION*, oferują urządzenia, które są nastawione na podobne cele. Należą do nich przede wszystkim obniżenie kosztów TCO (ang. *Total Cost of Ownership*), a także optymalizacja wydajności ruchu sieciowego w warstwach 4-7 modelu ISO/OSI. Omawiane produkty gwarantują skuteczną ochronę aplikacji internetowej, w szczególności przed atakami rozporoszonymi np. *DDOS*.

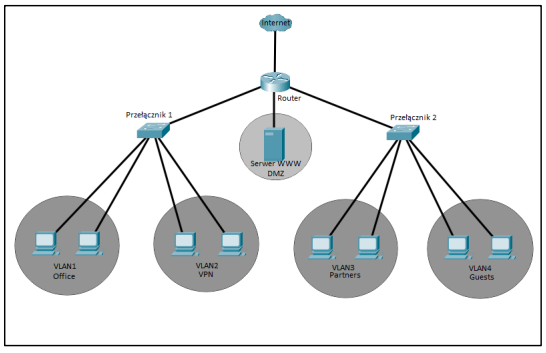
Rozdział III. Opracowanie projektu systemu zabezpieczeń

III.1. Opis istniejącej infrastruktury sieci komputerowej

Przed przystąpieniem do przygotowania wymagań na rozbudowywany system zabezpieczeń, należało poddać analizie istniejącą infrastrukturę sieciową firmy, dla której wykonywano zlecenie. Badanie to pozwoliło bliżej poznać istniejącą sieć teleinformatyczną, dzięki czemu ukształtowano kierunek działań potrzebnych do wdrożenia systemu zabezpieczeń.

Na potrzeby zadania przyjęto, że w siedzibie firmy zamawiającej istnieje już rozbudowana infrastruktura teleinformatyczna. Użytkownicy wspomnianej sieci mają dostęp do podstawowych usług, w tym dostęp do Internetu lub zdalny dostęp do danych przechowywanych na lokalnych serwerach.

Dotychczas klienci firmy dokonywali rezerwacji pojazdów telefonicznie. Firma podjęła decyzję o usprawnieniu tego procesu poprzez udostępnienie w sieci Internet aplikacji internetowej służącej do elektronicznego składania rezerwacji. Nowoutworzona aplikacja internetowa zostanie wdrożona na jednym z dotychczas wykorzystywanych serwerów WWW, podłączonych do strefy DMZ (ang. *Demilitarized Zone*). Na rysunku 2 zaprezentowano kluczowy, z punktu widzenia niniejszej pracy, wycinek sieci teleinformatycznej zamawiającego. Wyróżnione zostały cztery sieci VLAN (ang. *Virtual Local Area Network*) takie jak VLAN1, VLAN2, VLAN3 i VLAN4 oraz serwer WWW na którym zostanie aplikacja internetowa.



Rys. 2. Topologia sieci komputerowej firmy zamawiającego

Źródło: Opracowanie własne.

III.2. Założenia projektowe

Zarząd firmy określił cele, które chce osiągnąć po wdrożeniu nowej aplikacji internetowej. Celem priorytetowym jest udostępnienie możliwości rezerwacji pojazdów przez Internet oraz przegląd aktualnie dostępnych aut. W związku z tym, że wśród

przesyłanych informacji będą znajdowały się dane takie jak: dane osobowe klientów, historia rezerwacji pojazdów czy lista dostępnych pojazdów oraz ich wyposażenie, ważne jest zapewnienie odpowiedniego poziomu bezpieczeństwa wspomnianej aplikacji internetowej.

Nieautoryzowany dostęp do wspomnianych danych może przynieść za sobą poważne konsekwencje, do których należą straty wizerunkowe lub nawet finansowe firmy. Dodatkowo poszkodowani mogą być także klienci firmy, których dane mogą zostać skradzione.

III.3. Wymagania funkcjonalne

Firma zamawiająca system ochrony przed atakami sieciowymi przedstawiła swoje wymagania dotyczące wdrażanego rozwiązania. Wśród nich znalazły się wymagania funkcjonalne takie jak:

- system zabezpieczeń powinien zapewniać ochronę przed atakami z grupy *Injection*, w tym *SQL Injection* oraz *Cross-Site Scripting*,
- system zabezpieczeń powinien zapewniać ochronę przed atakami *DOS*,
- system zabezpieczeń powinien mieć możliwość konfiguracji w celu zwiększenia ochrony przed ciągle rozwijającymi się atakami sieciowymi,
- system zabezpieczeń powinien zapisywać informacje o każdym przeprowadzonym ataku na wdrożoną aplikację internetową,
- system zabezpieczeń nie powinien negatywnie wpływać na responsywność aplikacji internetowej,
- system zabezpieczeń nie powinien negatywnie wpływać na transmisję danych wewnątrz sieci lokalnej firmy.

III.4. Wymagania niefunkcjonalne

Oprócz opisanych w rozdziale III.3 wymagań funkcjonalnych, analiza wymagań firmy zamawiającej pozwoliła wyodrębnić również wymagania niefunkcjonalne. Wymagania dotyczące wdrażanego systemu są następujące:

- system zabezpieczeń powinien być dostępny przez cały czas, tzn. nie są dopuszczalne przerwy w dostarczaniu oferowanych funkcjonalności,
- system zabezpieczeń powinien być wdrożony na nowym urządzeniu sieciowym,
- system zabezpieczeń powinien znacząco ingerować w istniejącą infrastrukturę sieciową,
- system zabezpieczeń powinien być konfigurowalny z poziomu przeglądarki (aplikacja internetowa) oraz z poziomu wiersza poleceń.

Rozdział IV. Opracowanie projektu systemu

IV.1. Testy penetracyjne aplikacji internetowej przed wdrożeniem zabezpieczenia

Przed wdrożeniem docelowego systemu zabezpieczeń, przeprowadzono testy penetracyjne mające na celu wykazanie podatności dotychczas osadzonej na serwerze WWW aplikacji internetowej, tj. *InsecureWebApp*. Ta aplikacja internetowa została opracowana przez organizację *OWASP*, a mechanizmy w niej zaimplementowane celowo posiadają błędy i luki w zakresie zabezpieczeń. Z tego względu aplikacja ta z powodzeniem może być wykorzystywana m.in. do testów systemów ochrony sieci teleinformatycznej i przetwarzanych w niej zasobów. Omawiana aplikacja jest ponadto składnikiem środowiska treningowego *Web Security Dojo*, które zostało opracowane dla celów edukacyjnych w zakresie bezpieczeństwa aplikacji internetowych.

Testy podzielono na 4 części. Pierwszy etap polegał na skanowaniu aplikacji internetowej z wykorzystaniem oprogramowania *OWASP Zap*. Narzędzie umożliwia skanowanie aplikacji pod kątem występujących w niej adresów URL, funkcjonalności czy obsługiwanych parametrów, a następnie przeprowadza zbiór próbnych ataków na wymienione zasoby.

Etapy numer 2, 3 oraz 4 polegały na manualnym przeprowadzeniu ataków względem aplikacji internetowej. Dwa z nich dotyczyły ataków z grupy *Injection*, gdzie pierwszy to *SQL Injection*, a drugi to *Cross-Site Scripting*. Ostatni krok polegał na przeprowadzeniu ataku *DOS*.

IV.1.1. Testy penetracyjne przy wykorzystaniu aplikacji *OWASP Zap*

Aplikacje skanujące podatności są przydatne w przypadku sprawdzenia czy dana aplikacja internetowa jest zabezpieczona w akceptowalnym stopniu. Na rynku dostępnych jest wiele aplikacji skanujących (tzw. skanerów bezpieczeństwa), zaś jedną z rekomendowanych przez stowarzyszenie *OWASP* jest jego autorskie narzędzie *OWASP Zap*. Celem przeprowadzenia skanowania względem testowanej aplikacji, wystarczy wskazać wyłącznie docelowy adres URL witryny, która będzie podlegać badaniu pod kątem występujących w niej podatności. Szczegółowy raport skanowania aplikacji *InsecureWebApp* zaprezentowano na rysunku 3.



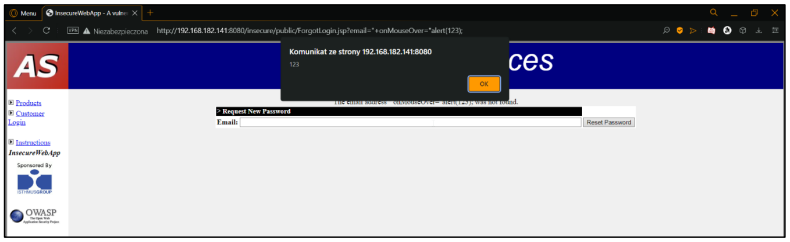
Rys. 3. Lista zagrożeń raportu programu *OWASP Zap*

Źródło: Opracowanie własne.

Na podstawie wygenerowanego raportu możliwe jest stwierdzenie na jakie ataki jest podatna testowana aplikacja internetowa. W tym przypadku aplikacja ta nie jest odporna na ataki z grupy *Injection*, a konkretnie *SQL Injection* oraz *Cross-Site Scripting*. W wygenerowanym raporcie prezentowane są także dokładne adresy URL, z użyciem których możliwe jest przeprowadzenie ataków wyżej wymienionych ataków.

IV.1.2. Przeprowadzenie ataku *Cross-Site Scripting*

Ataki *Cross-Site Scripting* należą do grupy ataków *Injection* i polegają na wstrzykiwaniu złośliwego kodu (najczęściej skryptu w języku *JavaScript*) do stron oraz aplikacji internetowych. W związku z tym przeprowadzono atak polegający na wstrzyknięciu złośliwego kodu do adresu URL celem wykonania skryptu po stronie przeglądarki klienta aplikacji internetowej. Spreparowany adres URL miał następującą postać:
http://192.168.182.200:8080/insecure/public/ForgotLogin.jsp?email="+onMouseOver="alert(123);, a wynik odwołania po ten adres przedstawiono na rysunku 4.



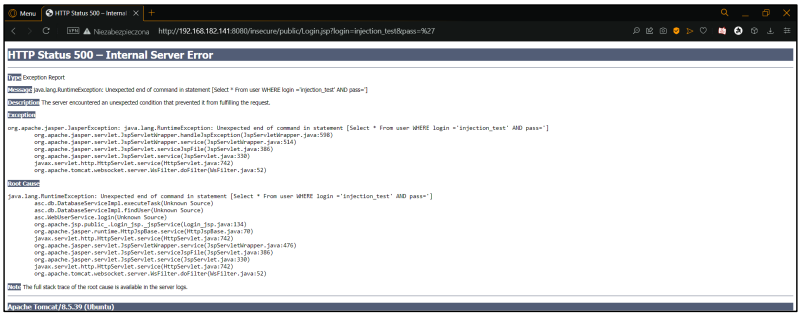
Rys. 4. Wynik działania ataku Cross-Site Scripting

Źródło: Opracowanie własne.

IV.1.3. Przeprowadzenie ataku *SQL Injection*

Przeprowadzenie ataku *SQL Injection* składa się z kilku kroków. Z raportu przedstawionego na rysunku 2 wynika, że na omawiany atak podatna jest witryna

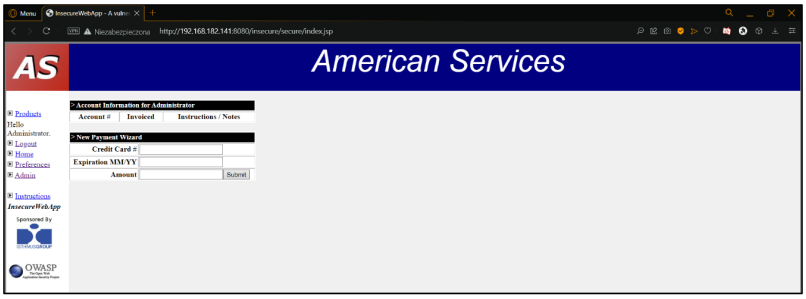
dostępna pod adresem *http://192.168.182.200:8080/insecure/public/Login.jsp*, a dokładniej formularz logowania do aplikacji internetowej. Pierwszy krok ataku stanowi zmuszenie serwera aplikacyjnego do tzw. rzucenia wyjątku i zwrócenia użytkownikowi śladu stosu (ang. *Stacktrace*). Do tej operacji wykorzystano dane uwierzytelniające w postaci nazwy użytkownika przyjmującej dowolny ciąg liter i cyfr oraz hasło w postaci znaku „*’* ”. Zwrócony w wyniku takiego odwołania błąd serwera przedstawiono na rysunku 5.



Rys. 5. Błąd serwera aplikacji

Źródło: Opracowanie własne.

Analizując przedstawiony na rysunku 4 stos wywołań, można określić w jaki sposób weryfikowane są dane logowania przez testowaną aplikację. Podczas tego procesu generowane jest zapytanie do bazy danych z wykorzystaniem polecenia: *Select * From user WHERE login ='injection_test' AND pass='*. Dokonując nieskomplikowanej modyfikacji wskazanego zapytania możliwe jest zalogowanie się na konto administratora bez konieczności wprowadzania poprawnego hasła użytkownika. Kolejny krok stanowi wyszukanie w bazie danych loginu konta administratora. Według artykułu WSTG - v4.2 [3] najczęstszymi nazwami użytkownika dla konta administratora są „*admin*”, „*administrator*” lub „*root*”. Wobec tego zapytanie SQL może przyjąć następującą postać: *Select * From user WHERE login ='admin’ --' AND pass=*, gdzie nazwa użytkownika to „*admin’ –*”. Wynik przeprowadzonego ataku zaprezentowano na rysunku 6.



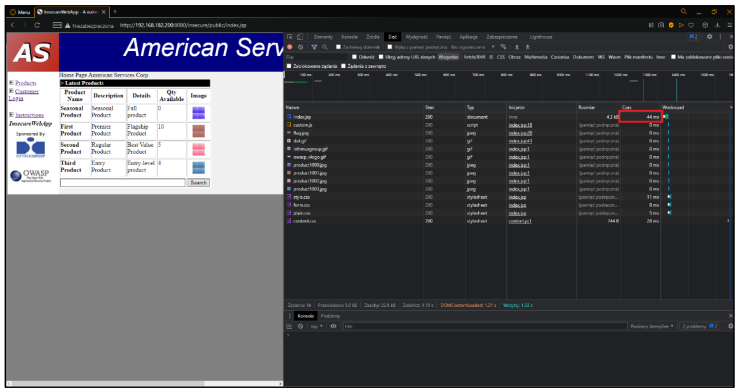
Rys. 6. Dostęp do aplikacji z konta administratora

Źródło: Opracowanie własne.

IV.1.4. Przeprowadzenie ataku *Denial Of Service*

Atak *DOS* polega na wysyłaniu wielu żądań HTTP jednocześnie celem zablokowania dostępu do aplikacji internetowej lub zmniejszenia jej responsywności. Do przeprowadzenia ataku wykorzystano program napisany w języku *Java*. Program powołuje do życia nowe wątki, które nieprzerwanie wysyłają żądania pod adres URL: *http://192.168.182.200:8080/insecure/public/?productSearch=abc*. W ramach żądania zostały ustawione manualne nagłówki żądania, takie jak: „*User-Agent*”, „*Content-Type*” czy „*Content-Length*”. Aplikacja, wykonująca atak, pracuje dopóki nie zostanie manualnie przerwany proces. Pierwszym krokiem było sprawdzenie jak długo trwa proces ładowania się witryny startowej testowanej aplikacji internetowej. Wynik przedstawiono na rysunku 7, a wyznaczony czas wczytywania witryny oznaczono kolorem czerwonym.

Z komentarzem [KD5]: Dopisałem o programie atakującym.

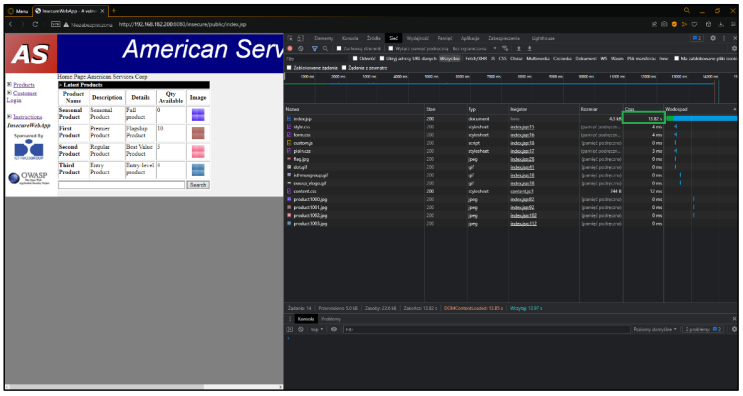


Rys. 7. Czas ładowania strony domowej aplikacji internetowej

Źródło: Opracowanie własne.

W kolejnym kroku uruchomiono program, który realizował atak *DOS* na aplikację internetową. W trakcie wykonywania testu, ponownie sprawdzono czas dostępu do

zasobów aplikacji internetowej. Uzyskany wynik przedstawiono na rysunku 8, zaś czas wczytywania strony internetowej zaznaczono kolorem zielonym.

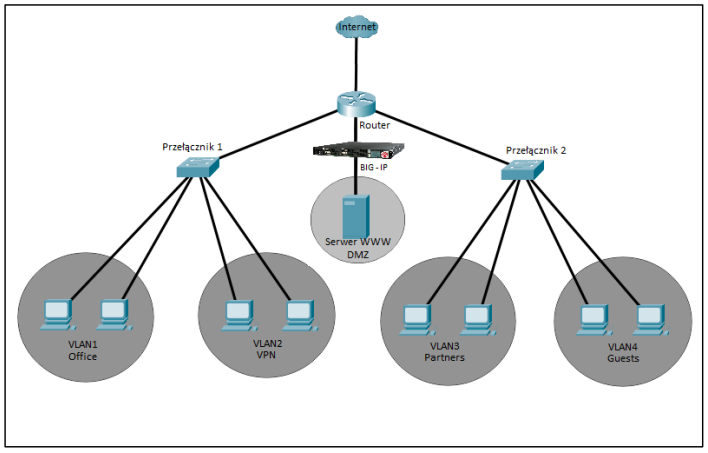


Rys. 8. Czas ładowania strony domowej aplikacji internetowej pod atakiem *DOS*
Źródło: Opracowanie własne.

IV.2. Projekt systemu ochrony

Projekt systemu zabezpieczeń serwera WWW, w tym aplikacji internetowej, bazuje na wykorzystaniu rozwiązania firmy F5 oferowanego w ramach produktu *BIG-IP*. Produkt ten składa się z dwóch elementów, które zapewniają skuteczną ochronę przed atakami sieciowymi. Jednym z nich jest *BIG-IP LTM* (ang. *Local Traffic Manager*) odpowiedzialny za kontrolę ruchu sieciowego wewnątrz danej infrastruktury teleinformatycznej. W ramach omawianego modułu dostępny jest mechanizm obsługi skryptów *iRules*, które pozwalają w pełni dostosować przepływ ruchu sieciowego według konkretnych potrzeb. Drugim elementem jest moduł *ASM* (ang. *Application Security Manager*). To rozwiązanie jest odpowiedzialne za blokowanie ataków sieciowych na aplikację internetową, w tym m.in. ataki z grupy *Injection* oraz ataki *DDOS* w warstwie aplikacji modelu *ISO/OSI*.

Na podstawie wymagań funkcjonalnych i niefunkcjonalnych zgłoszonych przez zamawiającą firmę, przygotowano uproszczony schemat infrastruktury sieciowej rozszerzonej o rozwiązanie *BIG-IP* (rys. 9). Omawiane urządzenie wdrożone zostanie w istniejącej infrastrukturze sieciowej w trybie *Reverse-Proxy* pomiędzy serwerem WWW a routerem brzegowym. Wdrożenie produktu we wskazanym miejscu umożliwia ukrycie serwera WWW wewnątrz sieci, przez co nie jest możliwe nawiązywanie z nim bezpośredniego połączenia.



Rys. 9. Topologia sieci komputerowej firmy zamawiającego rozbudowana o moduł *BIG-IP*
Źródło: Opracowanie własne.

Produkt *BIG-IP* jest możliwy do wdrożenia w dwóch wersjach - w wirtualnej i fizycznej. Wdrożenie wirtualne pozwala osadzić *BIG-IP* na istniejącym już urządzeniu oraz dopasować go do warunków, w których będzie on funkcjonował. Warto zaznaczyć, że produkt w wersji wirtualnej zapewnia te same funkcjonalności oraz ten sam stopień bezpieczeństwa co wersja fizyczna urządzenia.

W ramach realizowanego projektu, zaplanowano wdrożenie produktu *BIG-IP* w wersji wirtualnej. Przewidywany proces wdrożenia składa się z dwóch kroków realizowanych w odpowiedniej kolejności:

1. Konfiguracja maszyny wirtualnej z produktem *BIG-IP*.
2. Konfiguracja zasad bezpieczeństwa z poziomu interfejsu graficznego przy wykorzystaniu przeglądarki internetowej.

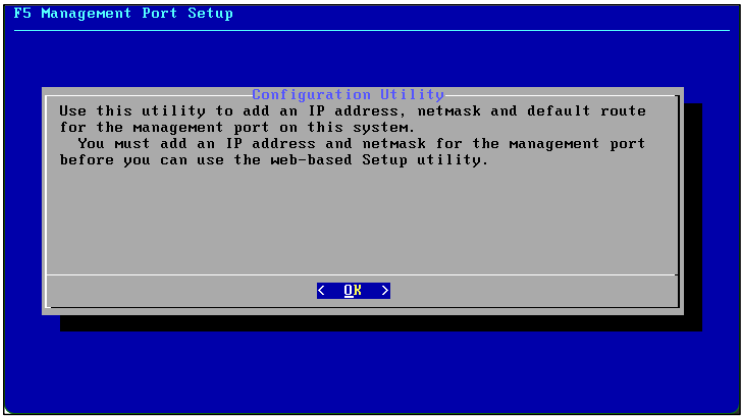
Do prawidłowego uruchomienia wirtualnej wersji produktu założono wykorzystanie środowiska *VMware Workstation*.

Rozdział V. Implementacja systemu

V.1. Konfiguracja maszyny wirtualnej

Wdrożenie systemu ochrony zabezpieczeń należy rozpocząć od zaimportowania obrazu maszyny wirtualnej do programu *VMware Workstation*. Pierwsze uruchomienie systemu, należy poprzedzić odpowiednim skonfigurowaniem interfejsów sieciowych zaimportowanej maszyny wirtualnej.

Kolejnym krokiem jest uruchomienie maszyny wirtualnej oraz zalogowanie się do systemu poprawnymi danymi uwierzytelniającymi (w przypadku *BIG-IP* są to *root/default*). Po pierwszym logowaniu do systemu należy zmienić hasło dostępowe do konta administracyjnego. W konsoli poleceń należy wykonać polecenie „*config*", które spowoduje uruchomienie narzędzia do konfiguracji interfejsów sieciowych maszyny wirtualnej (rys. 10).



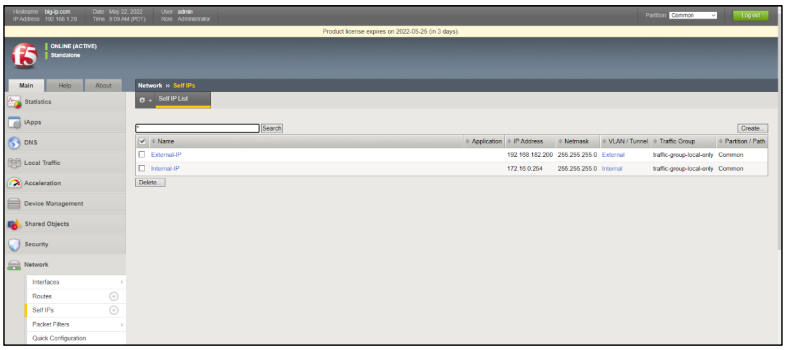
Rys. 10. Narzędzie konfiguracyjne parametrów sieciowych portu zarządzającego systemem
Źródło: Opracowanie własne.

Po wybraniu opcji „*OK*", w kolejnym etapie należy wybrać wersję konfiguracji „*IPv4*", a następnie zatwierdzić proponowane z serwera DHCP parametry takie jak adres sieciowy, maskę podsieci oraz adres bramy domyślnej.

V.2. Konfiguracja wstępna systemu

Otrzymany w wyniku wstępnej konfiguracji adres IP jest niezbędny, aby uzyskać dostęp do interfejsu konfiguracyjnego z poziomu przeglądarki internetowej. W tym celu należy nawiązać połączenie z adresem URL: *https://adres_serwera*. Następnie należy zalogować się do panelu administracyjnego serwera wykorzystując domyślne dane uwierzytelniające *admin*/*admin*. Produkt *BIG-IP* pozwala na zarządzanie swoimi interfejsami sieciowymi z poziomu graficznego interfejsu.

Pierwszym krokiem jest utworzenie dwóch sieci VLAN oraz przypisanie do nich odpowiednich interfejsów sieciowych maszyny wirtualnej. Pierwszy z nich odpowiedzialny jest za przyjmowanie ruchu sieciowego z sieci zewnętrznej, a drugi za przekierowanie go do sieci wewnętrznej. W tym celu należy przejść do menu „*Networks*", a następnie „*VLANs*". Po poprawnym utworzeniu sieci *VLAN* oraz przypisaniu do nich odpowiednich interfejsów sieciowych, należy utworzyć obiekty adresowe (sekcja *Self IP)* oraz podpiąć je do utworzonych wcześniej sieci *VLAN* (rys. 11). Obiekty utworzone w ten sposób umożliwiają komunikację między węzłami pracującymi w danej podsieci a rozwiązaniem *BIG-IP*.



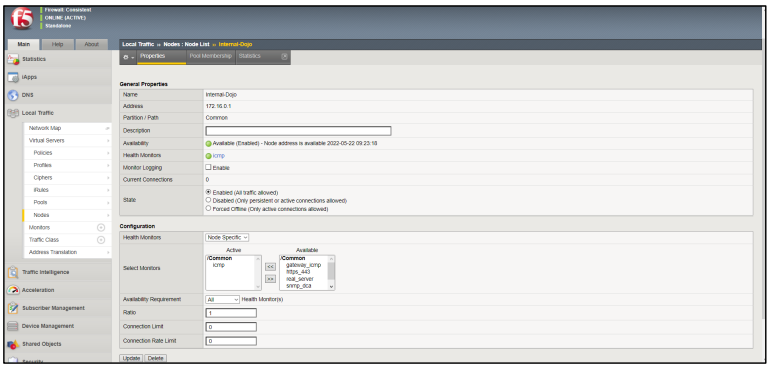
Rys. 11. Lista utworzonych adresów *Self IP*

Źródło: Opracowanie własne.

Wykonanie opisanych kroków konfiguracyjnych jest niezbędne do utworzenia połączenia między siecią zewnętrzną a siecią wewnętrzną DMZ, w której jest uruchomiony serwer WWW.

V.3. Konfiguracja polityk bezpieczeństwa

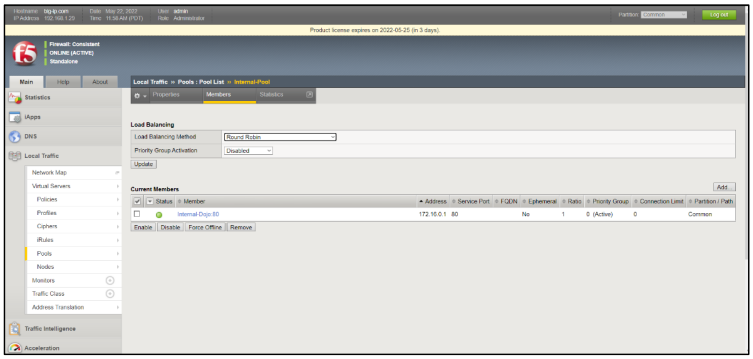
Kolejne kroki konfiguracyjne mają na celu dostosowanie systemu zabezpieczeń do postawionych wymagań. Czynnością do wykonania na typ etapie pracy jest dodanie serwera WWW do listy węzłów (ang. *Nodes*). Węzłem nazwany jest logiczny obiekt systemu BIG-IP, który reprezentuje adres IP fizycznego urządzenia. W zakładce „*Local Traffic*", a następnie „*Nodes*" należy utworzyć nowy węzeł podając adres IP serwera WWW, na którym wdrożona jest aplikacja internetowa.



Rys. 12. Właściwości węzła serwera WWW

Źródło: Opracowanie własne.

Utworzony zaś węzeł należy dodać do tzw. puli węzłów (ang. *Pools*). Obiekt ten jest możliwy do utworzenia w zakładce „*Local Traffic*", a następnie „*Pools*". Pulą nazywany jest zbiór urządzeń, w tym serwerów WWW, które są grupowane razem w celu odbierania i przetwarzania ruchu sieciowego. Rozwiązanie *BIG-IP* zamiast przekierowywać ruch klienta bezpośrednio do jednego hosta docelowego przekazuje żądanie do dowolnego węzła w puli zgodnie z wybranych mechanizmem równoważenia obciążenia. Należy przy tym zaznaczyć, że w omawianym środowisku teleinformatycznym członkiem puli będzie wyłącznie jeden węzeł, tj. serwer z aplikacją internetową *InsecureWebApp* (rys. 13).



Rys. 13. Lista członków puli

Źródło: Opracowanie własne.

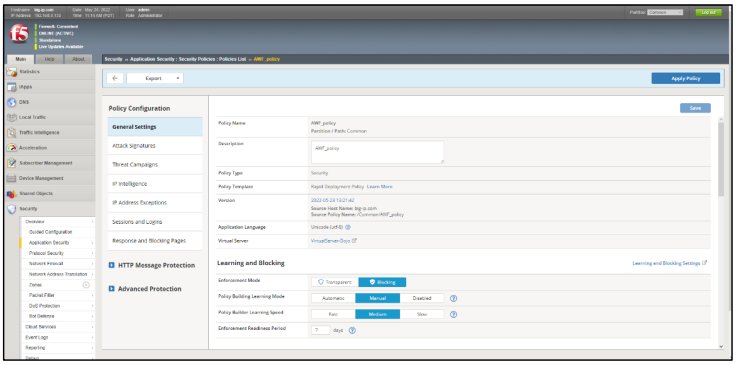
Posiadając skonfigurowany węzeł oraz pulę, celem kolejnego kroku jest utworzenie wirtualnego serwera. Omawiany obiekt jest jednym z ważniejszych elementów na etapie wdrażania produktu *BIG-IP*, ponieważ odpowiada za sterowanie ruchem sieciowym pomiędzy strefą zewnętrzną a wewnętrzną oraz za jego monitorowanie. Głównym

aspektem wirtualnego serwera jest rozdzielanie ruchu sieciowego między pulami, które zostają określone podczas konfiguracji. W celu utworzenia nowej instancji wirtualnego serwera, należy przejść do zakładki „*Local Traffic*", a następnie opcji „*Virtual Servers*". Podczas tworzenia nowego obiektu należy wskazać adres IP klienta „*Source Address*", czyli adres z którego będzie generowany przychodzący ruch sieciowy. Na etapie konfiguracji wskazany został zakres adresowy *0.0.0.0/0* co oznacza, że akceptowalny będzie ruch pochodzący z dowolnego węzła sieciowego. W polu „*Destination Address*" oraz „*Service Port*" wskazany został adres IP oraz port pod którym będzie możliwe połączenie z wirtualnym serwerem (od strony sieci zewnętrznej). W zakładce „*Resources*" utworzonego serwera wirtualnego należy wybrać pulę, która będzie obsługiwana przez ten serwer. Na tym etapie wirtualny serwer zostaje wstępnie skonfigurowany, co pozwala na korzystanie z aplikacji internetowej, do której ruch jest przekazywany przez rozwiązanie *BIG-IP*.

Kolejnym etapem jest wdrożenie polityki bezpieczeństwa, która gwarantuje ochronę aplikacji internetowej przed atakami sieciowymi. W tym celu wykorzystany zostanie moduł *BIG-IP ASM* (ang. *Application Security Manager*). Rozwiązanie *ASM* jest zaporą sieciową, która służy do zabezpieczania aplikacji internetowych. Pomaga chronić je przed podatnościami, które mogą wynikać np. z błędów implementacyjnych. W celu utworzenia nowej polityki bezpieczeństwa, należy wybrać z menu pozycję „*Security*", następnie „*Application Security*", a później „*Security Policies*". W tym miejscu widoczna jest lista wszystkich zdefiniowanych polityk bezpieczeństwa dla chronionych aplikacji internetowych. Warto zaznaczyć, że dla jednego wirtualnego serwera, może być aktywna tylko jedna polityka. Po kliknięciu przycisku „*Create*" użytkownikowi wyświetlony zostaje kreator tworzenia nowej polityki, w której należy zdefiniować elementy takie jak:

- nazwę polityki bezpieczeństwa – „*AWF_policy*",
- typ polityki – „*Security*",
- szablon polityki – „*Rapid Deployment Policy*",
- tryb egzekwowania – „*Blocking*",
- serwer wirtualny dla którego zostanie aktywowana polityka zabezpieczeń.

Szablon polityki „*Rapid Deployment Policy*" zapewnia gotowe funkcje zabezpieczeń, które są dostosowane do większości wymagań dotyczących bezpieczeństwa aplikacji internetowych, zmniejsza złożoność konfiguracji nowej polityki bezpieczeństwa, a co za tym idzie, zmniejsza czas wdrażania systemu zabezpieczeń. Oprócz tego, istotne jest, aby tryb egzekwowania działań ustawić na wartość „*Blocking*". W przeciwnym wypadku wszelkie wykryte ataki sieciowe na aplikacje internetową nie będą blokowane, a jedynie raportowane w dzienniku zdarzeń.



Rys. 14. Ekran konfiguratora polityki bezpieczeństwa aplikacji internetowej
Źródło: Opracowanie własne.

Skonfigurowany w ten sposób system ochrony zapewnia ochronę przed różnymi atakami sieciowymi, w tym, zgodnie z wymaganiami funkcjonalnymi, atakami *SQL Injection Cross-Site Scripting*. Z uwagi na temat niniejszej pracy zakłada się wykorzystanie skryptów *iRules* w celu ochrony przed wspomnianymi dwoma typami ataków oraz atakiem *DOS*. Wobec tego dla wdrożonej polityki bezpieczeństwa wyłączono predefiniowane sygnatury ataków dotyczące *SQL Injection*, *Cross-Site Scripting* oraz *DOS*.

Skrypty *iRules* są sterowane za pomocą zdarzeń. Oznacza to, że skrypt zostanie uruchomiony tylko w momencie, kiedy dane zdarzenie zostanie wykryte. Jednym z przykładów zdarzenia jest *HTTP_REQUEST*, co oznacza, że skrypt *iRule* korzystający z tego zdarzenia, zostanie uruchomiony za każdym razem kiedy system otrzyma żądanie HTTP. W ramach omawianych skryptów, dostępny jest zbiór poleceń, które wykonują określone czynności. Jednym z przykładów jest wyznaczanie trasy, przekierowywanie, modyfikowanie, opóźnianie, odrzucanie lub rejestrowanie ruchu sieciowego.

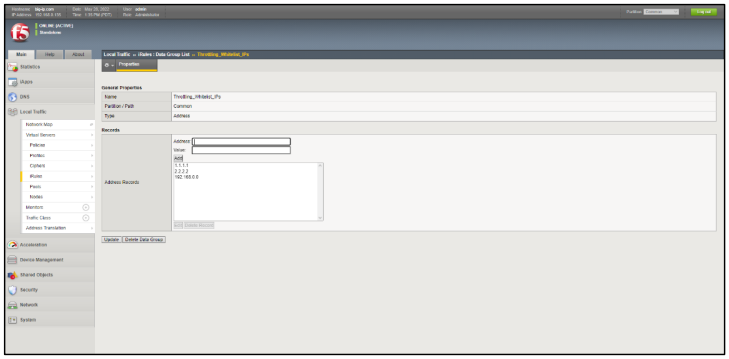
W miejsce wyłączonych sygnatur opracowano dedykowane skrypty *iRules*, które mają na celu wyeliminowanie wykrytych w rozdziale IV.1.2, IV.1.3 oraz IV.1.4 podatności. W raporcie wygenerowanym przez narzędzie *OWASP Zap* (rys. 3) wskazane zostały dokładne miejsca i adresy URL, w których aplikacja internetowa jest podatna na ataki z grupy *Injection*. Na podstawie analizy tego raportu stwierdzono, że atak *SQL Injection* jest możliwy do przeprowadzenia względem formularzy logowania i przywracania hasła, a złośliwy kod *SQL* jest przekazywany za pomocą parametrów żądania HTTP.

Na podstawie pozyskanych informacji przygotowano skrypt *iRule*, który weryfikuje zawartość adresu URL pod kątem występowania znaku apostrofu (bez niego nie jest możliwe wykonanie złośliwego kodu SQL w postaci sparametryzowanej) oraz najczęściej występujących ciągów znakowych w atakach *SQL Injection* [4]. Jeśli w adresie URL zostaną wykryte jakiekolwiek ze zdefiniowanych złośliwych wyrażeń,

zwrócony zostanie kod odpowiedzi *HTTP 200* oraz wyświetlona zostanie informacja o zablokowanym dostępie do zasobu aplikacji internetowej (załącznik 1.a).

W sposób analogiczny opracowano w skrypcie mechanizm wykrywania i blokowania ataków *Cross-Site Scripting*. W tym przypadku przechwytywane są najczęściej wykorzystywane w ataku *XSS* elementy HTML oraz funkcje *JavaScript*, w tym m.in. „*alert*", „*onload*", „*eval*", „*<script>*" (załącznik 1.a).

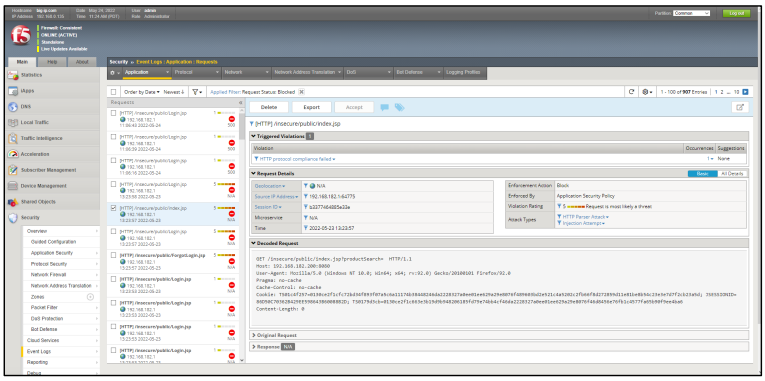
Skrypt odpowiedzialny za rozpoznawanie i blokowanie ataków *DOS* (załącznik 1.b) sprawdza ilość żądań HTTP przesłanych przez ten sam adres IP w określonym interwale czasowym. Dla wdrożonej aplikacji internetowej zezwolono na wysłanie 50 żądań HTTP w czasie 10 sekund. Jeśli limit żądań zostanie przekroczony, nadawcy zostanie zwrócona stosowana informacja, a żądanie HTTP nie zostanie przesłane do serwera WWW. Dodatkowo został zaimplementowany mechanizm, który sprawdza czy adres IP nie znajduje się na tzw. „białej liście". Jeśli adres znajduje się na wspomnianej liście to nie jest traktowany jako źródło ataków *DOS* i ma możliwość wysyłania nielimitowanej ilości żądań HTTP. Aby dodać adres IP do tej listy należy wybrać pozycję z menu „*Local Traffic*", następnie „*iRules*", a później „*Data Group List*". W tym miejscu należy utworzyć nowy zestaw danych o typie danych „*Address*" i dodać odpowiednie adresy IP do listy. Przykład takiej listy adresów został przedstawiony na rysunku 15.



Rys. 15. Lista adresów nie wliczanych do ataku *DOS*

Źródło: Opracowanie własne.

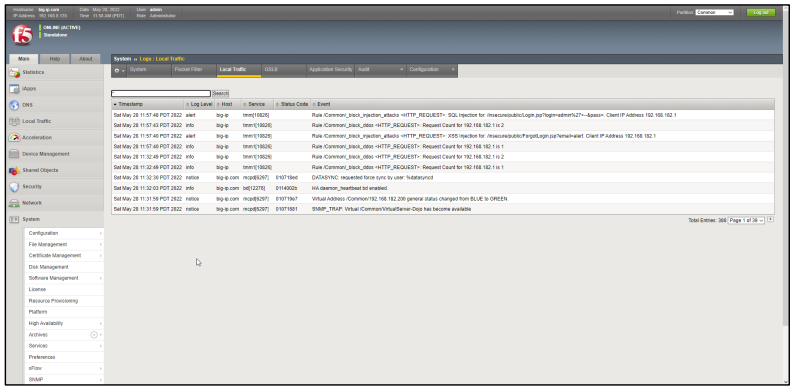
Jednym z założeń funkcjonalnych, które zostało zgłoszone przez firmę zamawiającą jest raportowanie wszelkich prób ataków sieciowych na aplikację internetową. Tego typu zdarzenia, wykryte przez produkt *BIG-IP*, są zapisywane w raporcie produktu, do którego wgląd mają administratorzy tego rozwiązania. W raporcie zapisywane są szczegółowe dane na temat zaistniałego zdarzenia, w tym: data oraz godzina zdarzenia, ID sesji, a nawet całe zapytanie REST (rys. 16).



Rys. 16. Ekran raportu produktu *BIG-IP*

Źródło: Opracowanie własne.

Z uwagi na sposób implementacji mechanizmu ochrony bazującego na skryptach *iRules*, raportowanie wykrywanych ataków zaimplementowano w sposób manualny. Informacje o tych atakach zapisywane są w pliku */var/log/ltm* na maszynie wirtualnej, na której zainstalowany jest produkt *BIG-IP*. Rekordy tego pliku są widoczne z poziomu panelu administracyjnego, w tym celu należy wybrać pozycję z menu „*System*”, następnie „*Logs*”, a później „*Local Traffic*” (rys. 17).



Rys. 17. Raport zablokowanych ataków z grupy Injection

Źródło: Opracowanie własne.

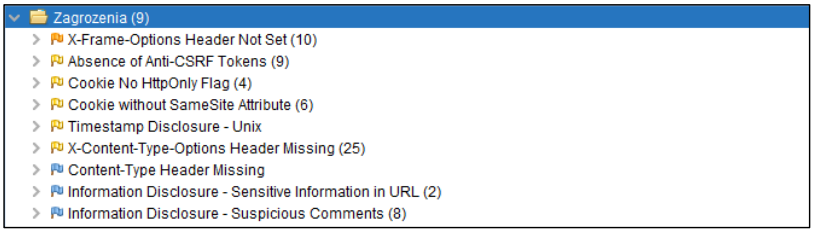
Rozdział VI. Testowanie systemu ochrony

Proces testowania jest istotny z punktu widzenia wdrożenia systemu zabezpieczeń. Podczas testowania możliwe jest sprawdzenie poprawności wdrożonego rozwiązania oraz możliwe jest wykrycie luk i błędów, które powstały w trakcie implementacji.

VI.1. Testy penetracyjne zabezpieczonej aplikacji internetowej przy wykorzystaniu aplikacji *OWASP Zap*

Z komentarzem [WK6]: Kursywą tak jak w treści rozdziałów i w całej pracy.

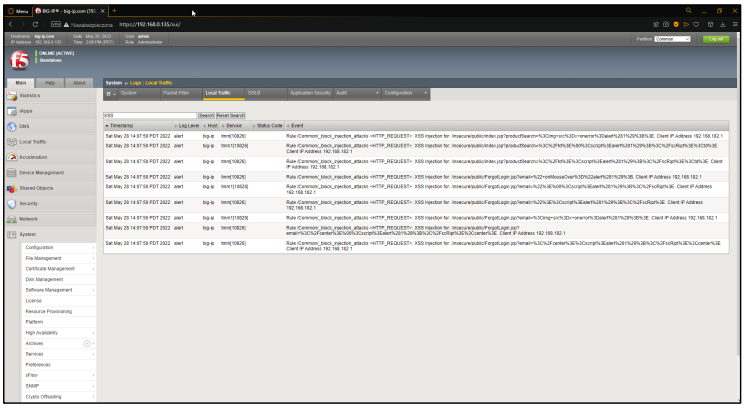
Pierwszym etapem w procesie testowania wdrożonego systemu ochrony jest przeprowadzenie testów penetracyjnych. W tym celu wykorzystano oprogramowanie *OWASP Zap*. Ten sam program został wykorzystany na etapie testowania aplikacji przed wdrożeniem systemu zabezpieczeń. Przeprowadzenie ponownego skanowania chronionej aplikacji internetowej umożliwiło uzyskanie aktualnego raportu, prezentującego listę wykrytych podatności w badanej aplikacji (rys. 18).



Rys. 18. Raport programu *OWASP Zap*

Źródło: Opracowanie własne.

Raport z przeprowadzonego skanowania nie wykazał występowania podatności z grupy *SQL Injection* oraz *Cross-Site Scripting*, które występowały przed wdrożeniem rozwiązania F5 *BIG-IP*. W trakcie testu i wykonywanych próbnych naruszeń, skrypt *iRules* skutecznie zablokował dostęp do aplikacji internetowej, a informacje o atakach zostały zapisane w pliku */var/log/ltm* na maszynie wirtualnej (rys. 19).

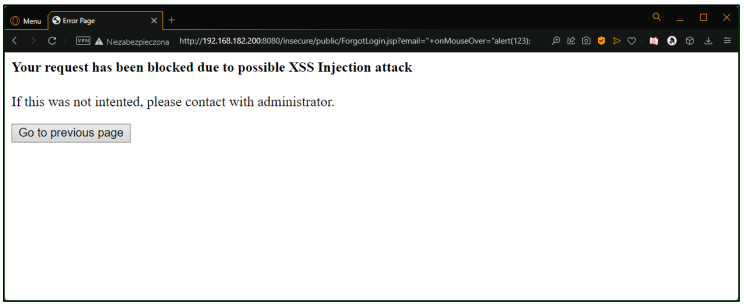


Rys. 19. Raport na temat ataków należących do grupy *Injection*

Źródło: Opracowanie własne.

VI.2. Przeprowadzenie ataku Cross-Site Scripting

Pierwszy z ataków, wykorzystuje podatności aplikacji internetowej należące do grupy *Cross-Site Scripting*. Atak ten ma na celu wykonanie złośliwego skryptu po stronie użytkownika, który po uruchomieniu witryny w przeglądarce internetowej zwróci użytkownikowi komunikat tekstowy. W tym celu wykorzystano ten sam, spreparowany adres URL, który został wykorzystany podczas testów przed wdrożeniem systemu ochrony. Jednakże tym razem przeprowadzenie tego ataku, zakończyło się niepowodzeniem, ponieważ został on skutecznie zablokowany przez skrypt *iRule*, a po stronie klienta został wyświetlony odpowiedni komunikat informacyjny (rys. 20).

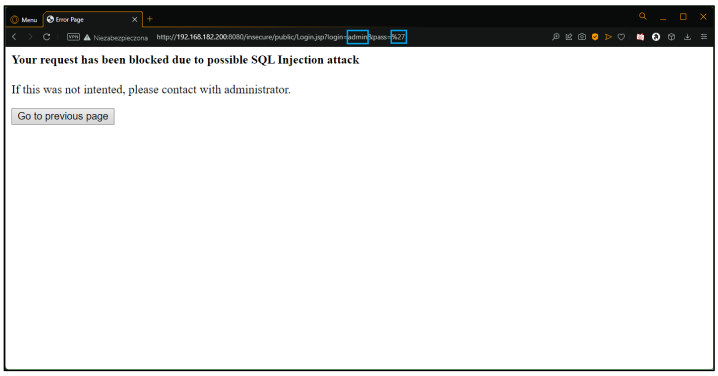


Rys. 20. Odpowiedź wygenerowana przez produkt *BIG-IP*

Źródło: Opracowanie własne.

VI.3. Przeprowadzenie ataku *SQL Injection*

Kolejny atak dotyczy wykorzystania podatności *SQL Injection*. Pierwszym krokiem, który należy wykonać podczas testu, jest próba wyświetlenia kodu błędu serwera po stronie klienta. W tym celu ponownie wykorzystano dane identyczne jak podczas testów wstępnych, gdzie w polu login wprowadzono wartość „*admin*", a w polu hasła znak „*'* ". Kluczowy ciąg znaków wprowadzony podczas wykonanej próby oznaczono kolorem niebieskim (rys. 21).

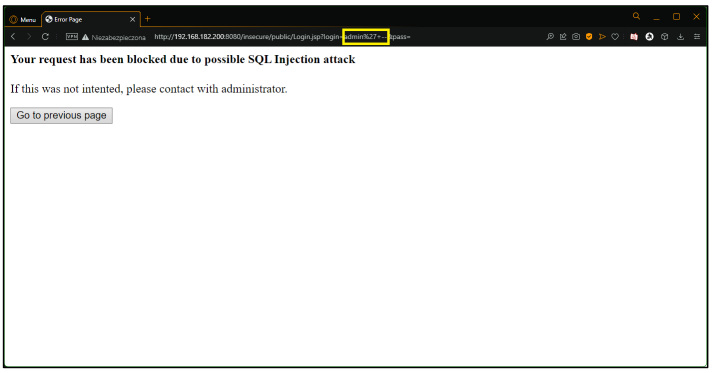


Rys. 21. Odpowiedź wygenerowana przez produkt *BIG-IP*

Źródło: Opracowanie własne.

Próba wyświetlenia śladu stosu aplikacji została przechwycona przez skrypt *iRule* i potraktowana jako potencjalny atak *SQL Injection*. Atak został skutecznie zablokowany.

Kolejny etap testów stanowi próba zalogowania się na konto administratora bez znajomości hasła. W tym celu, w polu login wprowadzono wartość „*admin* --", a w polu hasła dowolny ciąg znaków. Następnie przeprowadzono próbę uwierzytelnienia. Rezultat tej próby zaprezentowano na rysunku 22, a wprowadzone dane uwierzytelniające oznaczono kolorem żółtym.



Rys. 22. Odpowiedź wygenerowana przez produkt *BIG-IP*
Źródło: Opracowanie własne.

VI.4. Przeprowadzenie ataku *Denial of Service*

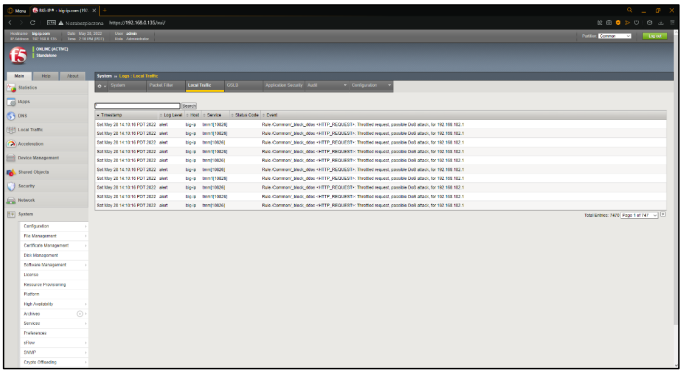
Ostatnim atakiem przeprowadzonym w ramach testów systemu ochrony jest atak *DOS*. W tym celu ponownie wykorzystano program napisany w języku *Java*. Podczas pracy programu wysyłającego wiele żądań HTTP w jednostce czasu dokonano próby załadowania strony domowej aplikacji internetowej. Uzyskany wynik przedstawiono na rysunku 23.

Z komentarzem [WK7]: Czy jest to autorski program, bo gdzieś wcześniej mi to umknęło. Jeśli nie to warto napisać o tym wzmiankę, a jeśli tak to warto go krótko opisać (w 2-3 zdaniach). Fragment z miejscem w którym trzeba uzupełnić opis opatrzyłem komentarzem w rozdziale z testami wstępnymi przed wdrożeniem systemu zabezpieczeń.



Rys. 23. Odpowiedź wygenerowana przez produkt *BIG-IP* na atak *DOS*
Źródło: Opracowanie własne.

Żądania HTTP, które zostały zakwalifikowane jako potencjalny atak *DOS* zostały zapisane w dzienniku zdarzeń *BIG-IP* (rys. 24).



Rys. 24. Raport przechwyconych ataków *DOS*

Źródło: Opracowanie własne.

Wnioski

Cel niniejszej pracy stanowiło opracowanie projektu systemu zabezpieczeń, wykorzystując reguły *iRules*. Przeprowadzona na początku pracy analiza zagrożeń, na które są podatne aplikacji internetowe, pozwoliła uporządkować wiedzę oraz uformować kierunek działań.

Na podstawie analizy produktów oraz rozwiązań oferowanych przez firmę F5 oraz potrzeby wykorzystania reguł *iRules* w niniejszej pracy, dokonano wyboru BIG-IP jako elementu głównego wdrażanego rozwiązania oraz dodatku w postaci modułu ASM.

Przeprowadzenie konfiguracji produktu *BIG-IP LTM*, który analizuje i filtruje ruch sieciowy, wdrożenie polityki bezpieczeństwa aplikacji wykorzystując moduł *ASM*, implementacja skryptów *iRules*, a także przeprowadzenie testów zabezpieczonej aplikacji pozwala uznać, że wdrożony system ochrony prawidłowo zabezpiecza aplikację przed atakami sieciowymi, na które jest ona podatna.

Warto zaznaczyć, że możliwość wprowadzania własnych sygnatur ataków, a także regularne aktualizacje ich bazy danych, znacząco wpływają na poziom bezpieczeństwa aplikacji. Ponadto skrypty *iRules* umożliwiają zarządzanie ruchem sieciowym według własnych potrzeb.

Interfejs graficzny produktu *BIG-IP* jest rozbudowany, jednak nie czyni go to trudnym w obsłudze z uwagi na jego intuicyjne działanie. Produkty firmy F5 posiadają obszerną dokumentację, która w znacznym stopniu ułatwia pracę z tym systemem.

Warto zwrócić uwagę, na różnorodność wdrożenia produktów *BIG-IP*. Wdrożenia wersji wirtualnej lub fizycznej urządzenia pozwalają na dostosowanie systemu do indywidualnych potrzeb. Należy zaznaczyć, że wersje wirtualne urządzenia, nie posiadają zmniejszonych funkcjonalności względem ich fizycznych odpowiedników, tym samym zapewniają aplikacji internetowej ten sam poziom bezpieczeństwa.

Nie należy jednak zapominać o ciągle rozwijających się metodach ataków sieciowych oraz innych zagrożeniach, na które mogą być narażone aplikacje internetowe. Nowopowstałe zagrożenia mogą być nieznane dla wdrożonego systemu zabezpieczeń, dlatego za zasadne uznaje się m.in. regularne monitorowanie ruchu sieciowego, co pozwoli dostatecznie szybko utworzyć nowe sygnatury ataków oraz dostosować system ochrony do zmieniających się warunków oraz potrzeb. Ponadto zaleca się cykliczne przeprowadzanie np. testów penetracyjnych, które poprzez realizację w kontrolowanych warunkach umożliwiają wykrycie podatności aplikacji internetowej bez wpływu na jej poprawność działania, a w konsekwencji jej zabezpieczenie przed potencjalnym atakiem.

Bibliografia

[1] http://owasp.org [dostęp: 10.02.2022].
[2] https://www.f5.com [dostęp: 07.03.2022].
[3] https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/04-Authentication_Testing/02-Testing_for_Default_Credentials [dostęp: 04.05.2022].
[4] https://help.deepsecurity.trendmicro.com/20_0/on-premise/intrusion-prevention-sql-injection.html [dostęp: 07.05.2022].
[5] https://en.wikipedia.org/wiki/Wikipedia:10,000_most_common_passwords [dostęp: 19.02.2022].
[6] https://clouddocs.f5.com/api/irules/ [dostęp: 05.05.2022].

Spis rysunków

Rys. 1. Schemat działania ataku DDoS 19
Rys. 2. Topologia sieci komputerowej firmy zamawiającego 25
Rys. 3. Lista zagrożeń raportu programu *OWASP Zap* 28
Rys. 4. Wynik działania ataku Cross-Site Scripting 28
Rys. 5. Błąd serwera aplikacji 29
Rys. 6. Dostęp do aplikacji z konta administratora 30
Rys. 7. Czas ładowania strony domowej aplikacji internetowej 30
Rys. 8. Czas ładowania strony domowej aplikacji internetowej pod atakiem *DOS* 31
Rys. 9. Topologia sieci komputerowej firmy zamawiającego rozbudowana o moduł *BIG-IP* 32
Rys. 10. Narzędzie konfiguracyjne parametrów sieciowych portu zarządzającego systemem 33
Rys. 11. Lista utworzonych adresów *Self IP* 34
Rys. 12. Właściwości węzła serwera WWW 35
Rys. 13. Lista członków puli 35
Rys. 14. Ekran konfiguratora polityki bezpieczeństwa aplikacji internetowej 37
Rys. 15. Lista adresów nie wliczanych do ataku *DOS* 38
Rys. 16. Ekran raportu produktu *BIG-IP* 39
Rys. 17. Raport zablokowanych ataków z grupy Injection 39
Rys. 18. Raport programu *OWASP Zap* 40
Rys. 19. Raport na temat ataków należących do grupy *Injection* 41
Rys. 20. Odpowiedź wygenerowana przez produkt *BIG-IP* 41
Rys. 21. Odpowiedź wygenerowana przez produkt *BIG-IP* 42
Rys. 22. Odpowiedź wygenerowana przez produkt *BIG-IP* 43
Rys. 23. Odpowiedź wygenerowana przez produkt *BIG-IP* na atak *DOS* 43
Rys. 24. Raport przechwyconych ataków *DOS* 44

Załączniki

1. Płyta CD/DVD zawierająca:
    a. Skrypt *iRule* blokujący ataki *SQL Injection* oraz *Cross-Site Scripting*,
    b. Skrypt *iRule* blokujący ataki *DOS*,
    c. Eksport konfiguracji produktu *BIG-IP*.